

ARC-AGI WITHOUT PRETRAINING IsaacLiao* AlbertGu CarnegieMellonUniversity CarnegieMellonUniversity iliao@andrew.cmu.edu agu@andrew.cmu.edu

ABSTRACT ConventionalwisdomintheageofLLMsdictatesthatsolvingIQ-test-likevisual puzzlesfromtheARC-AGI-1benchmarkrequirescapabilitiesderivedfrommas- sivepretraining. Tocounterthis,weintroduceCompressARC,a76Kparameter modelwithoutanypretrainingthatsolves20%ofevaluationpuzzlesbyminimizing thedescriptionlength(MDL)ofthetargetpuzzlepurelyduringinferencetime. TheMDLendowsCompressARCwithextremegeneralizationabilitiestypically unheardofindeeplearning. Toourknowledge,CompressARCistheonlydeep learningmethodforARC-AGIwheretraininghappensonlyonasinglesample;thetargetinferencepuzzleitself,withthefinalsolutioninformationremoved. More- over,CompressARCdoesnottrainonthepre-providedARC-AGI"trainingset". Undertheseextremelydata-limitedconditions,wedonotordinarilyexpectany puzzlestobesolvableatall. YetCompressARCstillsolvesadiversedistribution ofcreativeARC-AGIpuzzles,suggestingMDLtobeanalternativefeasiblewayto produceintelligence,besidesconventionalpretraining.

1 INTRODUCTION The ARC-AGI-1 benchmark consists of abstract visual reasoning puzzles designed to evaluate a system’s ability to rapidly acquire new skills from minimal input data (Chollet, 2019). Recent progress in LLM-based reasoning has shown impressive skill acquisition capabilities, but these systemsstillrelyonmassiveamountsofpretrainingdata. Inthispaper,weexplorehowlittledatais trulyrequiredtotackleARC-AGIbyintroducingCompressARC,asolutionmethodderivedfromthe MinimumDescriptionLength(MDL)principle(Rissanen,1978). CompressARCperformsallof itslearningatinferencetimeandachieves20%accuracyonARC-AGI-1evaluationpuzzles—using onlythepuzzlebeingsolvedasinputdata. ThekeytoCompressARC’sextremedataefficiencyisitsformulationasacode-golfingproblem: tofindtheshortestpossibleself-containedprogramthattoutpustheentireARC-AGIdataset,with anyunsolvedgridsfilledarbitrarily. ByOccam’srazor,wemayexpecttheshortestprogramtofill theunsolvedgridsinthemostsensibleway,whichiswith“correct”solutionsthatmatchtherules of the puzzle. An overly basic program might store a hard-coded string of the puzzle data (plus arbitrary solutions)foroutput;buttheprogramwillbetoolong,implyingtheoutputtedsolutions willbewrong. Ontheotherhand,findingtheoptimallyshortestprogramwouldrequireexhaustively enumerating many candidate programs, which is computationally infeasible (Solomonoff, 1964; Hutter,2005). CompressARCstrikesanewtypeofbalancebyoverfittinganeuralnetworktothe puzzle data to compress the puzzles into weight matrices; these weights can be hard-coded into the program instead of the puzzles themselves, and then used within the program to recover the memorizedpuzzleswithsolutions. Withcarefulcountingofthebitlengthofthehard-codedweights, thistechniqueconvertsthecombinatorialsearchoffindingthebestprogramintoadifferentiable optimizationproblem,allowingustominimizeprogramlengthinareasonableamountoftimeand stillgenerategoodsolutionpredictions. Thisframingpreservesseveralattractivepropertiesoftheoriginalcode-golf formulation,allofwhich arenovelwhenitcomestoneuralsolutionstoARC-AGI: • Nopretraining: Sincewebeginwiththetargetpuzzle(s)inhand,notrainingphaseisrequired. •https://iliao2345.github.io/ 1 5202 ceD 5]GL.sc[1v40160.2152:viXra

• Inference-time learning: Programlength isminimizedsolelyduringinference byoptimizing networkweightswithrespecttothetargetpuzzle(s). • Minimaldatarequirement: FollowingOccam’srazor,weassumestronggeneralizationfromthe shortestprogramanduseonlythepuzzle(s)themselves—noadditionaldataisloadedintomemory. Despiteneverusingthetrainingset,performingnopretraining,andhavingonly76Kparametersin itsnetwork,CompressARCgeneralizestosolve20%ofevaluationpuzzlesand34.75%oftraining puzzles—performancethatwouldbeimpossiblefortraditionaldeeplearningmethodsunderthese constraints. CompressARC’sstrongperformanceinthissettingsuggeststhatbringinginformation compressiontootherdata-limitedcontextsbeyondARC-AGI(e.g.,drugdiscovery,protein design) mayhelpuseextractstrongercapabilitiesinthoseapplicationsaswell. TheremainderofthispaperintroducestheARC-AGIbenchmark(Section2),detailstheproblem framing(Section3),describesCompressARC’sarchitecture(Section4),presentsempiricalresults (Section5),andconcludeswithadiscussionofimplications(Section6).

2 BACKGROUND: THE ARC-AGI BENCHMARK ARC-AGI-1isanartificialintelligencebenchmarkdesignedtotestsystem’sabilitytoacquirenew skillsfromminimalexamples(Chollet,2019). Eachpuzzleinthebenchmarkconsistsofadifferent hiddenrule,whichthesystemmustapplytoaninputcoloredgridtoproduceagroundtruthtarget coloredgrid. Thehiddenrulesmakeuseofthemeslikeobjectness,goal-directedness,numbers& counting,basicgeometry,andtopology. Severalinput-outputgridpairsaregivenasexamplestohelp thesystemfigureoutthehiddenruleinthepuzzle,andnootherinformationisgiven. Figure1shows threexamplesofARC-AGI-1trainingpuzzles. (a)Hiddenrule: Shifteveryob- (b)Hiddenrule: Shrinkthebig (c) Hidden rule: Extend the jectrightbyonepixel,exceptthe objectandsetitscolortothecat- greenlinetomeettheredlineby bottom/rightedgesoftheobject. tereddots’color. turningwhenhittingawall. Figure1: ThreexampleARC-AGI-1puzzles. WhilesolutionsbasedonLLMsbuiltusinginternetscaledatahavescored87.5%onthisbenchmark (Chollet,2024),andneuralnetworkstrainedononlyARC-AGIdatahavescored40.3% (Wangetal., 2025),CompressARCtakesthedata-minimalperspectivetoitlimits,optingtoonlytrainonthetest puzzle. AppendixLcontainsmoredetailsabouttheARC-AGI-1benchmark. Anexextendedsurveyofother relatedwork,includingotherapproachesforsolvingARC-AGI,isalsoincludedinAppendixA.Note thatwewillgenerallyrefertoARC-AGI-1justasARC-AGIinthispaper.

2

3 METHOD CompressARCtriestosolvetheproblemofcompressingthedataintoasshortaprogramaspossible, to obtain the puzzle solutions while keeping the program search feasible. In this case, the code mustbeentirelyself-contained,receivenoinputs,andmustprintouttheentireARC-AGIdataset ofpuzzleswithanyolutionsfilledin. Itistypicallyinfeasibletorunafullalgorithmicsearchto find the absolute shortest program, because we would have to search through a huge number of increasinglylengthyprogramstofindonewhoseprintoutmatchsourrequirements. CompressARC makesprogramsearchmoreamenablebyrestrictingitselftoasuitablywell-conditionedsubspaceof programs. Ouroverallstrategy,whichwewillexplaininmoredetailinthefollowingsections,will beasfollows: • Wedefineaspaceofprogramsthroughatemplateprogram(Algorithm1)whichcontainsnumerous emptyspacesforhardcodedvalues(“seeds”)tobeincluded. Thesehardcodedseeds willcomplete theprogram,sothatitcanberun,andtheprogramwillprintoutpuzzleswithsolutions. • Wedefineasearchalgorithm(Algorithm2)whichenumeratesmanypossibleseedstoplacein thesespaces,andselectsagoodsetofseedstomakeAlgorithm1asshortaspossible,withthe constraintthatitisoutputtedpuzzlesarethegivenpuzzles,butthesolutionsareunconstrained. Most ofthealgorithmlengthwillbetakenupbythelengthoftheseeds,soitsufficestoonlycountup thetotallengthoftheseedstoapproximatetheprogramlength. • OnlysomepartsofAlgorithm2arenecessarytooutputthesolution;therestareonlyinvolved ingeneratingtheactualcompressedrepresentationofthepuzzle. Tosolvethepuzzles,weonly keeptheformerparts,andweapproximatepartsoftheseearchwithagradientdescentprocedure (Algorithm3). Ourdifferentiableapproximationsofthevariousseedlengthswillbeanalogous totheindividualcomponentsofthelossfunctionforavariationalautoencoder(VAE)(Kingma& Welling,2022). Puzzle/solution data format: Each puzzle takes the form of a tensor of shape [n_example, width,height,2],containingcolordesignationsforeverypixelinthe2×n_examplegrids. Here,n_examplecountsthetotalnumberofinput/outputgridpairsinthepuzzle,includingthe testgridpairwhoseoutputgridisnotknown. Theshapeslistedinthissectionareforexplanatory purposesandtheactualdataformatisintroducedinSection4. Anaivefirsttryatcode-golfingthe datasetmayinvolve writingaprogramthathard-codeseachpuzzleinagiantstringandprintsit out. Improvementscanbemadebycleverwaysofde-duplicatingstructuresintheprinteddata(e.g., introducingforloops,etc.);wewilldetailourownparticularstrategybelow.

3.1 RESTRICTINGTHEPROGRAMSPACE

Webegin toderiveCompressARCby pickingaprogramsubspaceconsistingofatemplateprogram (Algorithm1) tobecompleted bysubstitutingvarioushard-codedvaluesintodesignatedlocations (shown in red). The template program generates each puzzle independently, and performs the operationsforeverypuzzle:

1. Itrandomlysamplesatensorzofshape[n_example,n_colors,width,height,2]from a standard normal distribution, (line 4) (There will be a sampling seed, whose length will be analogoustoaVAE’sKLloss term.)
 2. processesthiswithaneuralnetworkequivariant_NNwhichoutputs a[n_example,n_colors, width,height,2]-shapedlogittensor,(line6)
 3. and obtains a [n_example,width, height, 2]-shaped puzzle by sampling colors from the probability distribution implied by the logit tensor (line 8). We hope these colors match the givenpuzzle. (Therewillagainbeasamplingseed,whoselengthwillbeanalogoustoaVAE’s reconstructionloss term.) Theneuralnetworkequivariant_NNisaneuralnetworkthatis“equivariant”tovalidtransformations ofARC-AGIpuzzles. Thismeanswhentheinput/outputpairorderorcolorsarepermuted,oraflip orrotationisappliedtotheinputz,thentheoutputoftheneuralnetworkwillalsobetransformedin thesameway.
- 3**

Algorithm1:TemplateforashortprogramthatproducecompletedpuzzlesP withsolutions filled filledin. Redtextistobesubstitutedinwithhard-codedvaluesproducedviaAlgorithm2. Defineanequivariant_NNarchitecture; 1 2

3	Setseed_z=<seed_z	>;	HardcodedseedfromAlgo2,pu
1			
z ←sample	(N(0,1));		
-----	-----	---	----
4	seed_z		
Setθ =<θ	>;		HardcodedweightsfromAlgo2,
5	1		
6	grid_logits←equivariant_NN	(z);	
θ			
7	Setseed_error=<seed_error >;	HardcodedseedfromAlgo2,puzzle1	
1			
P ←sample	(grid_logits);		
-----	-----	-----	---
8	filled	seed_error	
PrintP			
9	filled		
-----	---	---	---
10			
11	Setseed_z=<seed_z	>;	HardcodedseedfromAlgo2,pu;
-----	---	---	---
2			
(...coderepeatsforallpuzzles) ...			
12			
The template program allows for two pseudo-random sampling seeds to be filled in for every puzzle			
(lines 3 and 7). The resulting printed puzzles can be guaranteed to match the true puzzles in the			
dataset by manipulating the final seed in line 7, which works after choosing any seed on line 3. With			
this guarantee in place, we can sum up the length of the code for the template, and find that the total			
length varies based on the number of bits/digits required to write down the two seeds.			
So, in order to			
search for short programs, we just need to make all these seeds as short as possible.			
Multiple areas of the program can be adjusted to help minimize these seed lengths, and we will cover			
each in respective sections: these seeds and the weights on lines 3, 5, and 7 (Section 3.2 below), and			
the architecture on line 1 (Section 4).			

3 Setseed_z=<seed_z		>;		HardcodedseedfromAlgo2,pu		
3.2 SEEDOPTIMIZATION						
Algorithm	2 presents	a method		of optimizing	the	
-----	-----	-----	---	-----	---	---
seedsandweightsintemplateAlgorithm1toreduce						
thetotalseedlength.		Itfirsttriestomanipulatethe				
-----	-----	-----	---	-----	-----	---
seed on	line 3 of	the template	to	imitate	z being	
sampledfromadifferentlearnednormaldistribution						
(line8),andthentriestomanipulatethesecondseed						
to guarantee	matching	puzzle	output	(line	11).	It
-----	-----	-----	-----	-----	-----	---
thenperformsgradientdescentonthenormaldistri-						
butionparametersandtheneuralnetworkweights						
tominimizethetotalseedlength(lines13-14).						
The idea	of “manipulating		the sampling		seed for	
-----	-----	-----	-----	---	-----	---
one distribution	to	imitate	sampling	from	another	
distribution”tendstobefraughtwithtechnicalities Figure2: CompressARCapproximatesaspe-						
cificcompressionalgorithmthatconvertsthe						
and subtleties.	Under	favorable		conditions,	this	
-----	-----	-----	---	-----	---	---
is cleanly achievable through rejection sampling ARC-AGIpuzzledatasetintotheshortestpro-						
(Forsythe, 1972), which produces a sample from gramthatprintsitoutexactly,alongwithany						
the imitated distribution using a seed whose ex- solutions.						
Theseprintedolutionsareassumed						
pected length is the max log probability ratio be- tobegoodpredictorsoftheactualsolutions,						
tweenothetwodistributions.			Thiscansubsequently		accordingtoOccam'srazor.	
-----	-----	-----	-----	-----	-----	-----
be improved	and	extended	towards	more	general	
conditions	using Relative		Entropy	Coding	(REC)	
(Harshaetal.,2010;Havasietal.,2018;Flamichetal.,2021),wheretheexpectedseedlengthlowers						
totheKLdivergencebetweenothetwodistributions,atthecostofonlyapproximatelyachievingthe						
desiredsamplingdistribution.			Wereferreaderstothesereferencesfordetails.			
-----	---	---	-----	---	---	---
Our main issue with seed manipulation using REC within Algorithm 2 on lines 8 and 11 is that						
runningRECrequiresexponentialtimeinthedivergencebetweenheimitateddistributionandthe						
samplingdistribution(Flamichetal.,2021). Sotomakeitfaster,weskipthesestepsandimitatetheir						
expecteddownstreamconsequencesinstead,resultinginCompressARC(Algorithm3).						
Namelyfor						
4						

Algorithm2:MinimizeDescriptionLength,a.k.a. code-golf. n_exampledenotesthetotal numberofinput/outputpairsinthepuzzle,includingthetestpairwheretheoutputisunknown. Line11: Thesmallestpossibleseed_errorispickedsothatthesampledpuzzleP filled online12 matchesthetruepuzzleP onboththeinputandoutputgridsofdemonstrationpairs,aswellas thetestinputs,butwithnorestrictiononthetestoutputs.

Input: ARC-AGIdataset;			
1			
Defineanequivariant_NNarchitecture;			
2			
foreachpuzzleP	inARC-AGIdatasetdo		
-----	-----	---	---
3			
4 Randomlyinitializeweightsθforequivariant_NN θ ;			
5 Observethedimensionsn_example,n_colors,width,heightofpuzzleP;			
Initializeinputdistribution: μofshape[n_example,n_colors,width,height,2],			
6			
anddiagonalΣ;			
foreachstepdo			
7			
8 Setseed_z,bymanipulatingtoimitatez			~N(μ,Σ);
-----	---	---	-----
9 z ←sample (N(0,I),shape=[n_example,n_colors,width,height,2]);			
seed_z			
grid_logits←equivariant_NN			(z);
-----	---	---	----
10			θ
Setseed_error,bymanipulatingtoobtaindesiredpuzzleP;			
11			
12 P	filled ←sample	(grid_logits);	

Input: ARC-AGIdataset;			
----	-----	-----	---
seed_error			
L←len(seed_z)+len(seed_error); ≈len(Algo1)+C			
13			
μ,Σ,θ	←Adam(∇	L,∇	L,∇ L);
----	-----	---	-----
14		μ Σ	θ
15 endforeach			
16 Insertvaluesseed_z,θ,andseed_errorintothepseudo-codeforAlgo1;			
endforeach			
17			
ReturncodeforAlgo1			
18			
line8,zisnowdirectlysampledfromtheimitateddistribution,andtheseedlengthfromline13is			
replacedbyitsexpectedlength,whichisveryclosetotheKLdivergencebetweentheimitatedand			
samplingdistributionsaccordingtothepropertiesofREC(Flamichetal.,2021). Forline11,the			
unknowngridsofthepuzzlearesampleddirectly,andtheseedlengthfromline13isapproximated			
closelybythenegativeloglikelihoodofsamplingtheknowngridsexactly,i.e.,thecross-entropy			
(seeAppendixBforderivation).			
Algorithm3:CompressARC.ItisthesameasAlgorithm2,butwithsimulatedseedmanipulation,			
andtruncatedtoreturnsolvedpuzzlesinsteadofdescription.			
1 Input: ARC-AGIdataset;			
-----	---	---	---
2 Defineanequivariant_NNarchitecture;			
foreachpuzzleP	inARC-AGIdatasetdo		
-----	-----	---	---
3			
Randomlyinitializeweightsθforequivariant_NN ;			
4 θ			
Observe dimensionsn_example,n_colors,width,heightofpuzzleP;			
5			
μofshape[n_example,n_colors,width,height,2],			
6 Initializeinputdistribution:			
anddiagonalΣ;			
foreachstepdo			
7			
z ←sample(N(μ,Σ));			
8			
grid_logits←equivariant_NN			(z);
-----	---	---	---
9			θ
L←KL(N(μ,Σ)		N(0,I))+cross-entropy(grid_logits,P); ≈len(Algo1)+C	
10			
11 μ,Σ,θ	←Adam(∇	μ L,∇ Σ	L,∇ θ L);
-----	-----	-----	-----
endforeach			
12			
P	←sample(grid_logits);		
---	-----	---	---
13 filled			
AddP	tosolvedpuzzles		
----	-----	---	---
14	filled		
15 endforeach			
Returnsolvedpuzzles			
16			
Algorithm3(CompressARC)isnowabletoautomaticallycompresstheARC-AGIdatasetthrough			
successiverefinementoftemplateAlgorithm1,outputtingsolutionsafterward. Theonlyremaining			
componenttospecifyistheneuralnetworkarchitectureusedwithintemplateAlgorithm1,whichwe			
5			

will design by hand. Since the architecture definition only appears once in Algorithm 1 while seeds appear repeatedly for every puzzle, using a sophisticated architecture can help us shorten the length of the template Algorithm 1, by trading off architecture description length to allow for shorter seeds. This serves as the primary motivation for us to heavily engineer the neural network architecture.

4 ARCHITECTURE

The central idea in designing the neural network architecture is to create a high probability of sampling the ARC-AGI puzzles, consequently reducing the length of these seeds and by extension the length of template Algorithm 1. According to the template structure, this means we need the neural network to have good inductive biases for transforming noise into reasonable-seeming ARC-AGI puzzles. The training puzzles play no role in our method other than to boost our effort to better engineer the inductive biases into our layers. Since ARC-AGI puzzles would be just as likely to appear in any combination of input/output example orderings, colors, orientations, etc., we want our network to assign them a equal probability by default. So, we made our architecture equivariant to example permutations, color permutations, rotations, and flips; (Cohen&Welling, 2016a) guaranteeing that computations applied to transformed inputs result in equivalent transformed outputs. For any asymmetries a puzzle may have, we require Algorithm 3 to manipulate the seed of the random input z , to bias the outputted puzzle one way or another.

Figure 3: Core structure of Compress ARC’s neural network, which operates on multiten-sor data. Individual operations (colored) read and write to a residual backbone through learned projections (grey) in the channel dimension. The network is equivariant to permutations of indices along the other, non-channel dimensions as a result. Some layers like

cummax break certain geometric symmetries,givingthearchitecture-specificgeometricabilitieslistedinAppendixH.Normalization, softmax,shift,anddirectionallayersarenotshown. Thearchitecture, showninFigure3, consists of a decoding layer functioning like an embedding matrix (details in Appendix D.1), followed by a core with a residual backbone, followed by a linear readout on the channel dimension (see Appendix D.8). In the core, linear projections on the channel dimension read data from the residual into specially designed operations, which write their outputs back into the residual through another projection. Normalization operations are scattered throughout the layers, and then the whole block of core layers is repeated 4 times. This is much like a transformer architecture, (Vaswani et al., 2023) except that the specially designed operations are not the attention and nonlinear operations on sequences, but instead the following operations on puzzle-shaped data:

- summing one tensor along an axis and/or broadcasting the result back up, (see Appendix D.2)
- taking the softmax along one or multiple axes of a tensor, (see Appendix D.3)
- taking a cumulative max along one of the geometric dimensions of a tensor, (see Appendix D.4)
- shifting by one pixel along one of the geometric dimensions of a tensor, (see Appendix D.4)

6

• elementwise nonlinearity, (see Appendix D.6)

• normalization along the channel dimension, (see Appendix D.7) along with one more described in Appendix D.5. The operations have no parameters and have their behaviors controlled by their residual read/write weights. All of these read/write projections operate on the channel dimension. We used 16 channels in some parts of the backbone and 8 in others to reduce computational costs. Since these projections take up the majority of the model weights, the entire model only has 76 K parameters. The actual data format that the neural network uses for computation is not a single tensor shaped like [n_example, n_colors, width, height, channel], but instead a bucket of tensors that each have a different subset of these dimensions, for example a [n_colors, width, channel] tensor. Both the input to the network and the outputted logits, as well as all of the internal activations, take the form of a multi-tensor. Generally, there is a tensor for every subset of these dimensions for storing information of that shape, which helps to build useful inductive biases. For example, an assignment of grid columns to colors can be stored as a one-hot table in the [color, width, channel]-shaped tensor. More detail on multi-tensors are in Appendix C.

5 RESULTS

We gave CompressARC 2000 inference-time training steps on every puzzle, taking about 20 minutes per puzzle. CompressARC correctly solved 20% of evaluation set puzzles and 34.75% of training set puzzles within this budget of inference-time compute. Figure 4 shows the performance increase as more inference-time compute is given. Tables 4 and 5 in the Appendix document the numerical solve accuracies with timings. Table 1: Comparison of solution accuracies of various methods for ARC-AGI-1, sorted by the amount of training data used. Each method makes two solution guesses per puzzle, and a guess is only correct if the grid shape and pixel colors are all correct. The U-Net (Ronneberger et al., 2015) baseline is a supervised model trained during inference time on only the demonstration input/output pairs of grids in the test puzzle to match the constraints of CompressARC; details in Appendix I. Method Trained on: Neural Acc. Dataset split Random guessing Nothing

Method	Train+test puzzles	✓	40.3% Public Eval	OpenAlo3high (Chollet, 2024)	Internet scaled data	✓	87.5% Semi-Priv. Eval	5.1 WHATPUZZLE CAN AND CAN'T WE SOLVE?
CompressARC	tries to use its abilities to figure out as much as it can, until it gets bottlenecked by one of its inabilities. For example, puzzle 28e73c20 in the training set requires extension of a pattern from the edge towards the middle, as shown in Figure 12a in the Appendix. Given the layers in its network, CompressARC is generally able to extend patterns for short ranges but not long ranges. So, it does the best that it can, and correctly extends the pattern a short distance before guessing at what happens near the center (Figure 12b, Appendix). Appendix H includes a list of which abilities we have empirically seen CompressARC able to and not able to perform.							

5.2 CASE STUDY: COLOR THE BOXES

In the puzzles shown (Figure 5), one must color the boxes depending on which side of the grid the box is on. We call this puzzle “Color the Boxes”. 7

(a) Training set of 400 puzzles. (b) Evaluation set of 400 puzzles. Figure 4: CompressARC’s puzzles solve accuracy rises as inference time learning progresses. Various numbers of allowed solution guesses (pass@n) for accuracy measurements are shown. The official benchmark is reported with 2 allowed guesses, which is why we report 20% on the evaluation set.

HumanSolution:	We first realize that the input is divided			
into boxes, and the boxes are still there in the output, but				
now they’re colored. We then try to figure out which colors				
go in which boxes.		First, we notice that the corners are		
-----	-----	---	-----	-----
always black.		Then, we notice that the middle is always		
magenta.		And after that, we notice that the color of the		
side boxes depends on which direction they are in:				red for
up, blue for down, green for right, and yellow for left.				At
this point, we copy the input over to the answer grid, then				
we color the middle box magenta, and then color the rest				
of the boxes according to their direction.				
CompressARC Solution: Table 2 shows CompressARC’s				
learning behavior over time.		After CompressARC is done		
-----	---	-----	---	---
learning, we can deconstruct its learned distribution to				
find that it codes for a color-direction correspondence table				
and row/column divider positions (Figure 7).				
During training,		the reconstruction	error fell	extremely
-----	-----	-----	-----	-----
quickly. It remained low on average, but would spike up				
z Figure 5: Color the Boxes,				
every once		in a while, causing	the KL from	to bump
-----	-----	-----	-----	-----
puzzle 272f95fa.				
upwards at these moments, as shown in Figure 6a.				
5.2.1 SOLUTION ANALYSIS				
We observe the representations stored in z to see how CompressARC learns to solve Color the Boxes.				
Since z is a multi-tensor, each of the tensors it contains produces an additive contribution to the total				
KL for z . By looking at the per-tensor contributions (see Figure 6b), we can determine which tensors				
in z code for information that is used to represent the puzzle.				
All the tensors fall to zero information content during training, except for four tensors. In some				
replication of this experiment, we saw one of these four necessary tensors fall to zero information				
content, and CompressARC typically does not recover the correct answers after that. Here we are				
showing a lucky run where the [color, direction, channel] tensor almost falls but gets picked				
up 200 steps in, which is right around when the samples from the model begin to show the correct				
colors in the correct boxes.				
We can look at the average output of the decoding layer (explained in Appendix D.1) corresponding				
to individual tensors of z , to see what information is stored there (see Figure 7). Each tensor contains				
a vector of dimension n_{channels} for various indices of the tensor. Taking the PCA of these				

Table 2: CompressARC learning the solution for Color the Boxes, over time. Learning

What is CompressARC doing?				Sampled solution guess
steps				
CompressARC's network outputs an answer				
grid (sample)	with light	blue	rows/columns	
-----	-----	-----	-----	-----
wherever	the input has	the same.	It	has no-
ticed that all the other input-output pairs in the				

What is CompressARC doing?			Sampled solution guess	
puzzle exhibit this correspondence.			It doesn't	
-----	---	---	-----	---
50 know how the other output pixels are assigned				
colors; an exponential moving average of the				
network	output (sample	average)	shows	the
-----	-----	-----	----	-----
network	assigning mostly	the	same	average
color on non-light-blue pixels.				
Then the network outputs a grid where near by pix-				
els have similar colors.			It has likely noticed	
-----	---	-----	---	---
150 that this is common among all the outputs, and				
is guessing that it applies to the answer too.				
Then the network output now shows larger blobs of				
colors that are cut off by the light blue borders.				
It has noticed the common usage of borders				
200 to demarcate blobs of colors in other outputs,			It has also no-	
and applies the same idea here.				
-----	---	---	-----	---
ticed black corner blobs in other given outputs,				
which the network imitates.				
Then the network output now shows the correct col-				
ors assigned to boxes of the correct direction				
from the center.	It has realized that a single			
-----	-----	---	---	---
color-to-direction mapping is used to pick the				
blob colors	in the other	given	outputs,	so it
-----	-----	-----	-----	-----
imitate this mapping.			It is still not the best	
350 at coloring within the lines, and it is also con-				
fused about the center blob, probably because				
the middle does not correspond to a direction.				
Nevertheless, the average network output does				
show a tinge of the correct magenta color in				
the middle, meaning the network is catching				
on.				
The network	is as refined	as	it will	ever be.
-----	-----	---	-----	-----
Sometimes it will still make a mistake in the				
1500				
sample outputs, but this is uncommon and fil-				
tered out.				
9				

(b) Breaking down the KL loss during training into (a) Relative proportion of the KL and reconstruction contributions from each individual shaped tensor to the loss during training, before taking in the multi-tensor z. Four tensors dominate, indicating they contain information, and the other 14 reconstruction is most often nearly perfect.

			fall to zero,		indicating their		lack of information	
content.								
Figure 6: Breaking down the loss components during training tells us where and how CompressARC								
prefer to store information relevant to solving a puzzle.								
vectors reveal some number of activated components, telling us how many pieces of information are								
coded by the tensor.								
(a) [example,		height,	channel]	and				
-----		-----	-----	---	---		---	---
[example,		width, channel]	tensors.	For				
every example and row/column, there is a vector of								
dimension n_{channels} . Taking the PCA of this								
set of vectors, the top principal component (> 1000								
(b) [direction, color, channel] tensor. A								
times stronger than the other components for both								
similar style PCA decomposition: the graph shows								
tensors) visualized as the [example,			height]					
-----		---	-----	---	---		---	---
the top principal component for this tensor. The								
and [example,		width]	matrices shown	above				
-----		-----	-----	----	---		---	---
four brightest pixels identify blue with up, green								

				fall to zero,	indicating their	lack of information
tells us which example/row and example/column						
				with left,	red with down,	and yellow with right.
-----	-----	-----	-----	-----	-----	-----
combinations	are uniquely	identified	by the			
This tells us each direction which color to						
store information. For every example, the two						
				use for the opposite edge's box.	The top princi-	
---	---	---	---	-----	---	-----
brightest pixels in the top matrix give positions						
pal component is 829 times stronger than the next						
of the light blue rows in the puzzle grids, and						
principal component.						
the darkest	two pixels	in the bottom	matrix			
-----	-----	-----	-----	---	---	---
indicate the columns.						
Figure 7: Breaking down the loss components during training tells us where and how CompressARC						
prefer to store information relevant to solving a puzzle.						
6 DISCUSSION						
The prevailing reliance of modern deep learning on high-quality data has put the field in a chokehold						
when applied to problems requiring intelligent behavior that have less data available. This is espe-						
10						

cially true for the data-limited ARC-AGI benchmark, where LLMs trained on specially augmented, extended, and curated datasets dominate (Knoop, 2024). In the midst of this circumstance, we built Compress ARC, which not only uses no training data at all, but forgoes the entire process of pretraining altogether. One should intuitively expect this to fail and solve no puzzles at all, but by applying MDL to the target puzzle during inference time, Compress ARC solves a surprisingly large portion of ARC-AGI-1. Compress ARC’s theoretical underpinnings come from minimizing the length of a programmatic description of the target puzzle. While other MDL search strategies have been scarce due to the intractably large search space of possible programs, Compress ARC explores a simplified, neural network-based search space through gradient descent. Though Compress ARC’s architecture is heavily engineered, its incredible ability to generalize from as low as two demonstration input/output pairs puts it in a entirely new regime of generalization for ARC-AGI. Efficiency improvement remains a valuable direction for future work on Compress ARC. Compress ARC makes use of many custom operations (See Appendices C and D), and adding JIT-compiled kernels or fused CUDA kernels would increase the training iteration speed. Improvements will naturally have a larger effect on larger grids since our architecture’s runtime scales with the number of pixels in the puzzle. We challenge the assumption that intelligence must arise from massive pretraining and data, showing instead that clever use of MDL and compression principles can lead to surprising capabilities. We use Compress ARC as a proof of concept to demonstrate that modern deep learning frameworks can be melded with MDL to create a possible alternative, complimentary route to AGI. 11

REFERENCES Ekin Akyürek, Mehul Damani, Adam Zweiger, Linlu Qiu, Han Guo, Jyothish Pari, Yoon Kim, and Jacob Andreas. The surprising effectiveness of test-time training for few-shot learning. arXiv preprint arXiv:2411.07279, 2024. ARC Prize Team. The hidden drivers of hrm’s performance on arc-agi. <https://arcprize.org/blog/hrm-analysis>, August 2025. Accessed: 2025-11-24. Guillermo Barbadillo. Solutions summary for arc24. https://ironbar.github.io/arc24/05_Solution_Summary/, 2024. Accessed: 2025-05-12. Guillermo Barbadillo. Exploring the combination of search and learn for the arc25 challenge, November 2025. URL https://ironbar.github.io/arc25/05_Solution_Summary/#introduction-what-is-arc-and-why-is-it-relevant. Accessed: 2025-11-22. Jeremy Berman. How i got a record 53.6% on arc-agi, December 2024. URL <https://jeremyberman.substack.com/p/how-i-got-a-record-536-on-arc-agi>. Blog post on Substack. Clément Bonnet and Matthew V MacFarlane. Searching latent program spaces, 2024. URL <https://arxiv.org/abs/2411.08706>. J. Bretnolle and C. Huber. Estimation des densités: Risque minimax. In C. Dellacherie, P.A. Meyer, and M. Weil (eds.), Séminaire de Probabilités XII, pp. 342–363, Berlin, Heidelberg, 1978. Springer Berlin Heidelberg. ISBN 978-3-540-35856-5. Vinod Kumar Chauhan, Jiandong Zhou, Ping Lu, Soheila Molaei, and David A. Clifton. A brief review of hypernetworks in deep learning. Artificial Intelligence Review, 57(9), August 2024. ISSN 1573-7462. doi: 10.1007/s10462-024-10862-8. URL <http://dx.doi.org/10.1007/s10462-024-10862-8>. François Chollet. On the measure of intelligence, 2019. URL <https://arxiv.org/abs/1911.01547>. François Chollet. Openai o3 breakthrough high score on arc-agi-pub. <https://arcprize.org/blog/oai-o3-pub-breakthrough>, 2024. Accessed: 2025-05-12. Taco Cohen and Max Welling. Group equivariant convolutional networks. In Maria Florina Balcan and Kilian Q. Weinberger (eds.), Proceedings of The 33rd International Conference on Machine Learning, volume 48 of Proceedings of Machine Learning Research, pp. 2990–2999, New York, New York, USA, 20–22 Jun 2016a. PMLR. URL <https://proceedings.mlr.press/v48/cohen16.html>. Taco S. Cohen and Max Welling. Group equivariant convolutional networks, 2016b. URL <https://arxiv.org/abs/1602.07576>. Jack Cole and Mohamed Osman. Don’t throw the baby out with the bathwater: How and why deep learning for arc. arXiv preprint arXiv:2506.14276, 2025. Shiqing Fan, Liu Liying, and Ye Luo. An alternative practice of tropical convolution to traditional convolutional neural networks, 2021. URL <https://arxiv.org/abs/2103.02096>. Gergely Flamich, Marton Havasi, and José Miguel Hernández-Lobato. Compressing images by encoding their latent representations with relative entropy coding, 2021. URL <https://arxiv.org/abs/2010.01185>. George E. Forsythe. Von neumann’s comparison method for random sampling from the normal and other distributions. Mathematics of Computation, 26(120):817–826, 1972. ISSN 00255718, 10886842. URL <http://www.jstor.org/stable/2005864>. Alex Graves, Greg Wayne, and Ivo Danihelka. Neural Turing machines, 2014. URL <https://arxiv.org/abs/1410.5401>. 12

Ryan Greenblatt. Getting 50% (sota) on arc-agi with gpt-4o. <https://redwoodresearch.substack.com/p/getting-50-sota-on-arc-agi-with-gpt>, 2024. Accessed: 2025-05-12. Prahladh Harsha, Rahu Jain, David McAllester, and Jaikumar Radhakrishnan. The communication complexity of correlation. IEEE Transactions on Information Theory, 56(1):438–449, 2010. doi: 10.1109/TIT.2009.2034824. Marton Havasi, Robert Peharz, and José Miguel Hernández-Lobato. Minimal random code learning: Getting bits back from compressed model parameters. arXiv preprint arXiv:1810.00440, 2018. Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition, 2015. URL <https://arxiv.org/abs/1512.03385>. Dan Hendrycks and Kevin Gimpel. Gaussian error linear units (gelus), 2023. URL <https://arxiv.org/abs/1606.08415>. Irina Higgins, Loic Matthey, Arka Pal, Christopher Burgess, Xavier Glorot, Matthew Botvinick, Shakir Mohamed, and Alexander Lerchner. beta-VAE: Learning basic visual concepts with a constrained variational framework. In International Conference on Learning Representations, 2017. URL <https://openreview.net/forum?id=Sy2fzU9gl>. Michael Hodel. Domain specific language for the abstraction and reasoning corpus. https://github.com/michaelhodel/arc-dsl/blob/main/arc_dsl_writeup.pdf, 2024. Accessed: 2025-05-12. Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models, 2021. URL <https://arxiv.org/abs/2106.09685>. Marcus Hutter. The Universal Algorithmic Agent AIXI, pp. 141–183. Springer Berlin Heidelberg, Berlin, Heidelberg, 2005. ISBN 978-3-540-26877-2. doi: 10.1007/3-540-26877-4_5. URL https://doi.org/10.1007/3-540-26877-4_5. Marcus Hutter. Hutter prize for lossless compression of human knowledge. <https://prize.hutter1.net/>, 2006. Accessed: 2025-05-12. Greg Kamradt. Arc prize 2024 solution: 4th place score 40. Kaggle Note-book, 2024. URL <https://www.kaggle.com/code/gregkamradt/arc-prize-2024-solution-4th-place-score-40-811b72>. Accessed: 2025-11-22. Diederik P Kingma and Max Welling. Auto-encoding variational bayes, 2022. URL <https://arxiv.org/abs/1312.6114>. Mike Knoop. ARC Prize 2024 Winners & Technical Report Published—arcprize.org. <https://arcprize.org/blog/arc-prize-2024-winners-technical-report>, 2024. [Accessed 12-05-2025]. A. N. Kolmogorov. On tables of random numbers. Theoretical Computer Science, 207(2):387–395, 1998. ISSN 0304-3975. doi: [https://doi.org/10.1016/S0304-3975\(98\)00075-9](https://doi.org/10.1016/S0304-3975(98)00075-9). URL <https://www.sciencedirect.com/science/article/pii/S0304397598000759>. G. G. Langdon. An introduction to arithmetic coding. IBM Journal of Research and Development, 28(2):135–149, 1984. doi: 10.1147/rd.282.0135. Solim LeGris, WaiKeen Vong, Brenden M. Lake, and Todd M. Gureckis. H-arc: A robust estimate of human performance on the abstraction and reasoning corpus benchmark, 2024. URL <https://arxiv.org/abs/2409.01374>. Wen-Ding Li, Keya Hu, Carter Larsen, Yuqing Wu, Simon Alford, Caleb Woo, Spencer M. Dunn, Hao Tang, Michelangelo Naim, Dat Nguyen, Wei-Long Zheng, Zenna Tavares, Yewen Pu, and Kevin Ellis. Combining induction and transduction for abstract reasoning, 2024a. URL <https://arxiv.org/abs/2411.02272>. 13

Wen-Ding Li, Keya Hu, Carter Larsen, Yuqing Wu, Simon Alford, Caleb Woo, Spencer M Dunn, Hao Tang, Michelangelo Naim, Dat Nguyen, et al. Combining induction and transduction for abstract reasoning. arXiv preprint arXiv:2411.02272, 2024b. Victor Vikram Odouard. Arc-solution_documentation. https://github.com/victorvikram/ARC-icecuber/blob/master/ARC-solution_documentation.pdf, 2024. Accessed: 2025-05-12. J. Rissanen. Modeling by shortest data description. Automatica, 14(5):465–471, 1978. ISSN 0005-1098. doi: [https://doi.org/10.1016/0005-1098\(78\)90005-5](https://doi.org/10.1016/0005-1098(78)90005-5). URL <https://www.sciencedirect.com/science/article/pii/0005109878900055>. Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-net: Convolutional networks for biomedical image segmentation. In International Conference on Medical Image Computing and Computer-Assisted Intervention, pp. 234–241. Springer, 2015. C. Shannon. The zero error capacity of a noisy channel. IRE Transactions on Information Theory, 2(3):8–19, 1956. doi: 10.1109/TIT.1956.1056798. C.E. Shannon. A mathematical theory of communication. The Bell System Technical Journal, 27(3): 379–423, 1948. doi: 10.1002/j.1538-7305.1948.tb01338.x. R. J. Solomonoff. A formal theory of inductive inference, part I. Information and Control, 7(1):1–22, 1964. ISSN 0019-9958. doi: [https://doi.org/10.1016/S0019-9958\(64\)90223-2](https://doi.org/10.1016/S0019-9958(64)90223-2). URL <https://www.sciencedirect.com/science/article/pii/S0019995864902232>. Yu Sun, Xiaolong Wang, Zhuang Liu, John Miller, Alexei A. Efros, and Moritz Hardt. Test-time training with self-supervision for generalization under distribution shifts, 2020. URL <https://arxiv.org/abs/1909.13231>. Casper Kaae Sønderby, Tapani Raiko, Lars Maaløe, Søren Kaae Sønderby, and Ole Winther. Ladder variational autoencoders, 2016. URL <https://arxiv.org/abs/1602.02282>. Alexandre B. Tsymbakov. Introduction to nonparametric estimation. Springer Publishing Company, Incorporated, 1st edition, 2008. ISBN 0387790519. Arash Vahdat and Jan Kautz. Nvae: A deep hierarchical variational autoencoder, 2021. URL <https://arxiv.org/abs/2007.03898>. Aaron van den Oord, Oriol Vinyals, and Koray Kavukcuoglu. Neural discrete representation learning, 2018. URL <https://arxiv.org/abs/1711.00937>. Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need, 2023. URL <https://arxiv.org/abs/1706.03762>. Guan Wang, Jin Li, Yuhao Sun, Xing Chen, Changling Liu, Yue Wu, Meng Lu, Sen Song, and Yasin Abbasi-Yadkori. Hierarchical reasoning model. arXiv preprint arXiv:2506.21734, 2025. Ruibin Xiong, Yunchang Yang, Di He, Kai Zheng, Shuxin Zheng, Chen Xing, Huishuai Zhang, Yanyan Lan, Liwei Wang, and Tie-Yan Liu. On layer normalization in the transformer architecture, 2020. URL <https://arxiv.org/abs/2002.04745>. Manzil Zaheer, Satwik Kottur, Siamak Ravanbakhsh, Barnabas Poczos, Ruslan Salakhutdinov, and Alexander Smola. Deepsets, 2018. URL <https://arxiv.org/abs/1703.06114>. 14

A RELATED WORK A.1 EQUIVALENCE OF COMPRESSION AND INTELLIGENCE The original inspiration of this work came from the Hutter Prize (Hutter, 2006), which awards a prize for those who can compress a file of Wikipedia text the most, as a motivation for researchers to build intelligent systems. It is premised upon the idea that the ability to compress information is equivalent to intelligence. This equivalence between intelligence and compression has a long history. For example, when talking about intelligent solutions to prediction problems, the ideal predictor implements Solomonoff Induction, a theoretically best possible but uncomputable prediction algorithm that works universally for all prediction tasks (Solomonoff, 1964). This prediction algorithm is then equivalent to a best possible compression algorithm whose compressed code length is the Kolmogorov Complexity of the data (Kolmogorov, 1998). This prediction algorithm can also be used to decode a description of the data of minimal length, linking these formulations of intelligence to MDL (Rissanen, 1978). In our work, we try to approximate this best possible compression algorithm with a neural network. A.2 INFORMATION THEORY AND CODING THEORY Since we build an information compression system, we make use of many results in information theory and coding theory. The main result required to motivate our model architecture is the existence of Relative Entropy Coding (REC) (Flamich et al., 2021). The fact that REC exists means that as long as a KL divergence can be bounded, the construction of a compression algorithm is always possible and the issue of realizing the algorithm can be abstracted away. Thus, problems about coding theory and translating information from Gaussians into binary and back can be ignored, since we can figure out the binary code length directly from the Gaussians instead. In other words, we only need to do enough information theory using the Gaussian to get the job done, with no coding theory at all. While the existence of arithmetic coding (Langdon, 1984) would suffice to abstract the problem away when distributions are discrete, neural networks operate in a continuous space so we need REC instead. Our architecture sends information through an additive white Gaussian noise (AWGN) channel, so the AWGN channel capacity formula (Gaussian input Gaussian noise) plays a heavy role in the design of our decoding layer (Shannon, 1948). A.3 VARIATIONAL AUTOENCODERS The decoder side of the variational autoencoder (Kingma & Welling, 2022) serves as our decompression algorithm. While we would use something that has more general capabilities like a neural Turing machine (Graves et al., 2014) instead, neural Turing machines are not very amenable to gradient descent-based optimizations so we stuck with the VAE. VAEs have a long history of developments that are relevant to our work. At one point, we tried using multiple decoding layers to make a hierarchical VAE decoder (Sønderby et al., 2016) instead. This does not affect the KL calculation because each channel capacity with feedback is equal to the channel capacity without feedback (Shannon, 1956). But, we found empirically that the first decoding layer would absorb all of the KL contribution, making the later decoding layers useless. Thus, we only used one decoding layer at the beginning. The beta-

VAE(Higginsetal.,2017)introducesareweightingofthereconstructionlosstobestronger thantheKLloss,andwefoundthattoworkwellinourcase. TheNVAEappliesanon-constant weighting to loss components (Vahdat & Kautz, 2021). A rudimentary form of scheduled loss recombinationisusedinCompressARC. A.4 OTHERARC-AGIMETHODS Top-scoringmethodstosolveARC-AGIrelyonconvertingpuzzlegridsintotextandthenfeeding themintoapretrainedlargelanguagemodelwhichispromptedtofindthesolution. Thepredominant techniquesinvolveeitherusingtheLLMtooutputthesolutiongriddirectly(Lietal.,2024b;Cole 15

&Osman,2025;Akyüreketal.,2024),oroutputaprogramthatcanberuntomanipulatethegrids instead(Li et al., 2024b; Greenblatt, 2024; Barbadillo, 2025; Berman, 2024). Oftentimes, these methodsemployseveraltrickstoimproveperformance: • Fine-tuningontrainingpuzzledata – Applyingdataaugmentationtoincreasetheeffectivenumberofpuzzlestofine-tuneon(Akyürek etal.,2024) – Fine-tuningonsyntheticdata(Lietal.,2024a;Akyüreketal.,2024) • Employinginference-timettrainingapproaches – Fine-tuninganindividualmodelspecifictoeachtestpuzzle,duringtesttime(Akyüreketal., 2024) – Test-timettraining(TTT)techniques(Sunetal.,2020;Barbadillo,2024) • Samplingmanymodeloutputsorrandomaugmentationsofthetestpuzzle,forensembling(Cole& Osman,2025;Greenblatt,2024) • LLMreasoning(Chollet,2024) Suchmethodshavemanagedtoscoreupto87.5%onthesemi-privatesplitofARC-AGI,atacost ofover\$200equivalentofinference-timecomputeperpuzzle(Chollet,2024). Theseapproaches allmakeuseoflanguagemodelsthatwerepretrainedontheentireinternet,whichisincontrastto CompressARC,whoseonlytrainingdataisthetestpuzzle. TheircommonalitieswithCompressARC aremainlyintheemphasisontrainingindividualmodelsonindividualtestpuzzlesandtheuseof ensemblingtoimprovesolutionpredictions. Asidefromthesemethods,severalothermethodshavebeenstudied: • Anolderclassofmethodsconsistsofhard-coded,large-scalesearchesthroughprogramspacesin hand-writtendomain-specificlanguagesdesignedspecificallyforARC(Hodel,2024;Odouard, 2024). Whilethesemethodsdonotusenaturalnetworkstosolvepuzzlesandarelessinstructive towardssthefieldofmachinelearning,theysharethecommonalityofusingheavilyengineered componentsdesignedspecificallyforARC-AGI. • (Bonnet&Macfarlane,2024)introducedaVAE-basedmethodforsearchingthroughalatentspace ofprograms. ThisisthemostsimilarworktooursthatwefoundduetotheirVAEsetup. A.5 DEEPLARNINGARCHITECTURES Wedesignedourownneuralnetworkarchitecturefromscratch,butnotwithoutborrowingcrucial designprinciplesfrommanyothers. Ourarchitectureisfundamentallystructuredlikeatransformer,consistingofaresidualstreamwhere representationsarestoredandoperatedupon,followedbyalinearhead(Vaswanietal.,2023;He etal.,2015). Pre-andpost-normswithlinearp-anddown-projectionsallowlayerstoreadandwrite totheresidualstream(Xiongetal.,2020). TheSiLU-basednonlinearlayerisespeciallysimilartoatransformer’s(Hendrycks&Gimpel,2023). Our equivariance structures are inspired by permutation-invariant neural networks, which are a typeofequivariantneuralnetwork(Zaheeretal.,2018;Cohen&Welling,2016b). Equivariance transformationsaretakenfromcommonaugmentationstoARC-AGIpuzzles. B SEED LENGTH ESTIMATION BY KL AND CROSS-ENTROPY InSection3.1,weestimatelen(seed_z)inline12oftemplateAlgorithm2askL(N(μ,Σ)||N(0,I)), andlen(seed_error)ascross-entropy(grid_logits,P). Inthissection,wewillargueforthereason- abilityofthisapproximation. ReadersmayalsorefertoFlamichetal. (2021),whichintroducesa betterseedmanipulationmethodas“RelativeEntropyCoding”(REC).Flamichetal.(2021)shows that seed communication is effectively the most bit-efficient way for an encoder and decoder to communicatesamplesfromadistributionQifthereisasharedsourceofrandomnessP. Thisuses nearly KL(P||Q) expected bits per sample communicated. We urge readers to refer to Flamich 16

etal.(2021)fordetailsregardingthemanipulationprocedure,runtimeandmemoryanalysis,and approximationstrength. Below,wefollowwithourowneffortatreasoningthroughwhy. Torecap,weoriginalprocedureinAlgorithm2manipulatestheseedforsamplingz ~N(0,I)to simulateasthoughz ~N(μ,Σ),andwewouldliketoshowthatwecancloselyapproximatethis samplingusinganexpectednumberofseedbitsclosetoKL(N(μ,Σ)||N(0,I)). For sake of illustration, suppose for instance that Algorithm 2 implements something similar to rejectionssampling,(Forsythe,1972)iteratingthroughseedsonbyoneandacceptingthesample withprobabilitymin(1,cw(z))forsomec<1,wherew(z)istheprobabilityratio N(z;μ,Σ) w(z)= N(z;0,I) Whenwepickasmallenoughc,thesamplingdistributionbecomesarbitrarilyclosetoN(μ,Σ)as desired. Withthisc,wewouldliketoshowthattheexpectednumberofrejectionsleadstosustoendup withaseedlengthclosetotheKL. WewouldfirstliketolowerboundtheprobabilityP accept ofacceptingateachstep,whichis (cid:90)

	P		=		N(z;0,l)min(1,cw(z))dz	
accept						
			(cid:90)		(cid:18) cN(z;μ,Σ)	(cid:19)
---	---		-----		-----	---
			= N(z;0,l)min		1,	dz
N(z;0,l)						
(cid:90)						
			= min(N(z;0,l),cN(z;μ,Σ))		dz	
---	---		-----		---	---
WewillfollowamodifiedversionofaderivationoftheBretagnolle–Huberinequality(Bretagnolle&						
Huber,1978)byTsybakov(2008)toderiveaboundontheKL:						
(1+c)P	≥(1+c–P		)P			
-----	-----		-----		-----	-----
	accept		accept		accept	
	(cid:18)(cid:90)		(cid:19)(cid:18)(cid:90)		(cid:19)	
	=		dz		min(N(z;0,l),cN(z;μ,Σ)) dz	
	max(N(z;0,l),cN(z;μ,Σ))					
whereapplyingtheCauchy-Schwarzinequalitywithafunctionspaceinnerproduct,						
	(cid:18)(cid:90)		(cid:19)2			
---	-----		---		-----	---
(cid:112)						
≥ max(N(z;0,l),cN(z;μ,Σ))min(N(z;0,l),cN(z;μ,Σ))dz						
	(cid:18)(cid:90)		(cid:19)2			
---	-----		---		-----	---
(cid:112)						
= cN(z;0,l)N(z;μ,Σ)dz						
	(cid:18)		(cid:90)		(cid:19)	
---	---		-----		-----	---
(cid:112)						
	=cexp		2ln	N(z;0,l)N(z;μ,Σ)dz		
---	-----		-----	-----		---
	(cid:32)		(cid:115)		(cid:33)	
			(cid:90)		N(z;0,l)	
	=cexp		2ln	N(z;μ,Σ)		dz
N(z;μ,Σ)						
	(cid:32)		(cid:34)(cid:115)		(cid:35)(cid:33)	
---	---		-----		-----	---
N(z;0,l)						
2lnE						
=cexp						
			z~N(z;μ,Σ)		N(z;μ,Σ)	
---	---		-----		-----	---
andfollowingwithJensen’sinequality,						
			(cid:18)		(cid:20) N(z;0,l)	(cid:21)(cid:19)
---	---		---		-----	-----
E						
	≥cexp		z~N(z;μ,Σ)		ln	

P		=		N(z;0,l)min(1,cw(z))dz	
---	---	---	----	-----	---
N(z;μ,Σ)					
=cexp(−KL(N(μ,Σ		N(0,l)))			
leadstoanacceptanceprobabilityofatleast					
c					
P		≥ exp(−KL(N(μ,Σ		N(0,l)))	
---	---	---	----	-----	---
accept		1+c			
17					

Therefore,accordingtotherejectionsamplingprocedure,thenumberofsamplesproposed(i.e. the expectedseed)isatmosttheinverseofthisacceptanceprobability, 1+c seed_zs exp(KL(N(μ,Σ)||N(0,1))) c sotheexpectedseedlengththisatmostaroundthelogarithm, len(seed_z)≤KL(N(μ,Σ)||N(0,1))+log(1+c)−logc matchingupwithourstatedKLapproximationoftheseedlength. For the seed_error term, Algorithm 2 manipulates the seed to sample a puzzle P from a dis- tribution implied by some logits. This is effectively the same as sampling grid_logits ~ Categorical_distribution(logits) and manipulating the seed to try to get grid_logits ~ Delta_distribution(P). Then,thesameKL- basedboundonrequiredseedlengthcanbeusedonce again. Theexpectedseed_errorlengththisatmost KL(Delta_distribution(P)||Categorical_distribution(logits))+log(1+c)−logc whichsimplifiesas (cid:20) (cid:21) δ(x=P) E log +log(1+c)−logc x~Delta_distribution(P) Categorical_probability(x;logits) δ(P =P) =log +log(1+c)−logc Categorical_probability(P;logits) =−logCategorical_probability(P;logits)+log(1+c)−logc =cross_entropy(logits,P)+log(1+c)−logc wheretheδis1whenthestatementwithinistrue,and0otherwise. C MULTITENSORS Theactualdata(z,hiddenactivations,andpuzzles)passingthroughourlayerscomesinaformatthat wecalla”multitensor”,whichisjustabucketoftensorsofvariousshapes,asshowninFigure8. Alltheequivariancesweusecanbedescribedintermsofhowtheychangeamultitensor. Figure 8: Our neural network’s internal representations come in the form of a “multitensor”, a bucketoftensorsofdifferentshapes. Oneofthetensorsisshapedlike[example,color,height, width,channel],anadequateshapeforstoringawholeARC-AGIpuzzle. Mostcommonclassesofmachinelearningarchitecturesoperateonasingletypeoftensorwithcon- stant rank. LLMs operate on rank-3 tensors of shape [n_batch, n_tokens, n_channels], and Convolutional Neural Networks (CNNs) operate on rank-4 tensors of shape [n_batch, n_channels, height, width]. Our multitensors are a set of varying-rank tensors of unique type, whose dimensions are a subset of a rank-6 tensor of shape [n_example, n_colors, 18

n_directions,height,width,n_channels],asillustratedinFigure8. Wealwayskeep thechannelldimension,sothereareatmost32tensorsineachmultitensor. Wealsomaintainseveral rules(seeAppendixE.1)thatteterminewhetheratenorsorshapeis”legal”ornot,whichreduces the numberoftensorsinamultitensorto18. Dimension Description Example Number of examples in the ARC-AGI puzzle, including the one with held-out answer Color NumberofuniquecolorsintheARC-AGIpuzzle, notincludingblack, seeAp- pendixF.2 Direction 8 Height Determinedwhenpreprocessingthepuzzle,seeAppendixF.1 Width Determinedwhenpreprocessingthepuzzle,seeAppendixF.1 Channel Intheresidualconnections,thissizeis8ifthedirectiondimensionisincluded, else16. Withinlayersitislayer-dependent. Table3: Sizeconventionsformultitensordimensions. Togiveanideaofhowamultitensorstoresdata,anARC- AGIpuzzlecanberepresentedbyusing the[example,color,height,width,channel]tensor,byusingthechannelldimension to select either the input or output grid, and the height/width dimensions for pixel location, a one hot vector in the color dimension, specifying what color that pixel is. The [example, height, channel] and [example, width, channel] tensors can similarly be used to store masksrepresentinggridshapesforeveryexampleforeveryinput/outputgrid. Allthosetensorsare includedinasinglemultitensorthatiscomputedbythenetworkjustbeforethefinallinearhead (describedinAppendixD.8). Whenweapplyanoperationonamultitensor,webydefaultassumethatallnon-channeldimen- sionsaretreatedidenticallyasbatchdimensionsbydefault. Theoperationiscopiedacrosstheindices ofdimensionsunlessspecified. Thisensures thatwekeepalloursymmetriesintactuntilweusea specificlayermeanttobreakaspecificsymmetry. Afinalnoteonthechannelldimension: usuallywhentalkingaboutatenors’sshape,wewillnot evenmentionthechannelldimensionasisincludedbydefault. D LAYERS IN THE ARCHITECTURE D.1 DECODINGLAYER Thislayer’sjobistosampleamultitensorandbounditsinformationcontent,beforeitispassed tothenextlayer. ThislayerandoutputstheKLDivergencebetweenthelearnedzdistributionand N(0,l). PenalizingtheKLpreventsCompressARCfromlearningadistributionforzthatmemorizes theARC-AGIpuzzleinanuncompressedfashion,andforcesCompressARCtorepresentthepuzzle moresuccinctly. Specifically,itforcesthenetworktospendmorebitsontheKLwheneveritusesz tobreakasymmetry,andthelargerthesymmetrygroupbroken,themorebitsspend. Thislayertakesasinput: • Alearnedtargetmultiscalar,calledthe”targetcapacity”. 1 Thedecodinglayerwilloutputzwhose informationcontentpertensorisclosetothetargetcapacity,2 • learnedper-elementmeansforz,3 • learnedper-elementcapacityadjustmentsforz. 1Targetcapacitiesareexponentiallyparameterizedandrescaledby10toincrease sensitivitytolearning, initializedataconstant104natspertensor,andforcedtobeaboveaminimumvalueofhalfanat. 2Theactualinformationcontent,whichthelayercomputeslateron,willbeslightlydifferentbecauseofthe per-elementcapacityadjustments. 3Meansareinitializedusingnormaldistributionofvariance10−4. 19

We begin by normalizing the learned per-element means for z.4 Then, we figure out how much Gaussiannoise we must add into every tensor to make the AWGN channel capacity (Shannon, 1948) equal to the target capacity for every tensor (including per- element capacity adjustments). We apply the noise to sample z, keeping unit variance of z by rescaling.5 We compute the information content of z as the KL divergence between the distribution of this sample and N(0, 1). Finally, we post process the noisy z by scaling it by the sigmoid of the signal-to-noise ratio.6 This ensures that z is kept as-is when its variance consists mostly of useful information and it is nearly zero when its variance consists mostly of noise. All this is done 4 times to make each channel dimension of 4. Then we apply a projection (with different weights per tensor in the multitensor, i.e., per-tensor projections) mapping the channel dimension up to the dimension of the residual stream. D.2 MULTITENSOR COMMUNICATION LAYER This layer allows different tensors in a multitensor to interact with each other. First, the input from the residual stream passes through a per-tensor projection to a fixed size (8 for downwards communication and 16 for upwards communication). The message is sent to every other tensor that has at least the same dimensions for upwards communication, or at most the same dimensions for downwards communication. This message is created by either taking means along dimension to remove them, or unsqueezing + broadcasting dimension to add them, as in Figure 9. All the messages received by every tensor are summed together and normalization is applied. This result gets up- projected back and then added to the residual stream. Figure 9: Multitensor communication layer. Higherrank tensors show at the top, lower rank at the bottom. Tensor transform between ranks by mean reduction and unsqueezing dimensions. D.3 SOFTMAX LAYER This layer allows the network to work with internal one-hot representations, by giving it the tools to denoise and sharpen noisy one-hot vectors. For every tensor in the input multitensor, this layer lists out all the possible subsets of dimensions of the tensor to take a softmax over,7 takes the softmax 4 Means and variances for normalization are computed along all non-channel dimensions. 5 There are many caveats with the way this is implemented and how it works; please refer to the code (see Appendix O) for more details. 6 We are careful not to let the post processing operation, which contains unbounded amounts of information via the signal-to-noise ratios, to leak a lot of information across the layer. We only let a bit of it leak by averaging the signal-to-noise ratios across individual tensors in the multitensor. 7 One exception: we always include the example dimension in the subset of dimensions. 20

over these subset of dimensions, and concatenates all the softmaxed results together in the channel dimension. The output dimension varies across different tensors in the multitensor, depending on their tensor rank. A pre-norm is applied, and per- tensor projections map to and from the residual stream. The layer has an input channel dimension of 2. D.4 DIRECTIONAL CUMMAX / SHIFT LAYER The directional cummax and shift layers allow the network to perform the non-equivariant cummax and shift operations in a equivariant way, namely by applying the operations once per direction, and only letting the output be influenced by the results on the other directions are aggregated back together (by the multitensor communication layer). These layers are there solely because we included the direction dimension when defining a multitensor: to store the results of directional layers and operate on each individually. Of course, this means when we apply a spatial equivariant transformation, we must also permute the indices of the direction dimension accordingly, which can get complicated sometimes. The directional cummax layer takes the eight indices of the direction dimension, treats each slice as corresponding to one direction (4 cardinal, 4 diagonal), performs a cumulative max in the respective direction for each slice, does it in the opposite direction for half the channels, and stacks the slices back together in the direction dimension. An illustration is in Figure 10. The slices are rescaled to have min − 1 and max 1 before applying the cumulative max. The directional shift layer does the same thing, but for shifting the grid by one pixel instead of applying the cumulative max, and without the rescaling. Some details: • Per-tensor projections map to and from the residual stream, with pre-norm. • Input channel dimension is 4. • These layers are only applied to the [example, color, direction, height, width, channel] and [example, direction, height, width, channel] tensors in the input multitensor. Figure 10: The directional cummax layer takes a directional tensor, splits it along the direction axis, and applies a cumulative max in a different direction for each direction slice. This operation helps Compress ARC transport information across long distances in the puzzle grid. D.5 DIRECTIONAL COMMUNICATION LAYER By default, the network is equivariant to permutations of the eight directions, but we only want symmetry up to rotations and flips. So, this layer provides a way to send information between two slices in the direction dimension, depending on the angular difference in the two directions. This 21

layer defines a separate linear map to be used for each of the 64 possible combinations of angles, but the weights of the linear maps are minimally tied such that the directional communication layer is equivariant to reflections and rotations. This gets complicated really fast, since the direction dimension’s indices also permute when equivariant transformations are applied. Every direction slice in a tensor accumulates its 8 messages, and adds the results together.8 For this layer, there are per- tensor projections to and from the residual stream with pre-norm. The input channel dimension is 2. D.6 NONLINEAR LAYER We use a SiLU nonlinearity with channel dimension 16, surrounded by per-tensor projections with pre-norm. D.7 NORMALIZATION LAYER We normalize all the tensors in the multitensor, using means and variances computed across all dimensions except the channel dimension. Normalization as used within other layers also generally operates this way. D.8 LINEAR HEADS We must take the final multitensor, and convert it to the format of an ARC-AGI puzzle. More specifically, we must convert the multitensor into a distribution over ARC-AGI puzzles, so that we can compute the log-likelihood of the observed grids in the puzzle. Figure 11: The linear head layer takes the final multitensor of the residual stream and reads a [example, color, height, width, channel] tensor to be interpreted as color logits, and a [example,height,channel] tensor and a [example,width,channel] tensor to serve as shape masks. The color of every pixel for every example for both input and output, have logits defined by the [example, color, height, width, channel] tensor, with the channel dimension linearly mapped down to a size of 2, representing the input and output grids.9 The log-likelihood is given by the cross-entropy, with sum reduction across all the grids. 8 We also multiply the results by coefficients depending on the angle: 1 for 0 degrees and 180 degrees, 0.2 for 45 degrees and 135 degrees, and 0.4 for 90 degrees. 9 The linear map is initialized to be identical for both the input and output grid, but isn’t fixed this way during learning. Sometimes this empirically helps with problems of inconsistent input vs output grid shapes. The bias on this linear map is multiplied by 100 before usage, otherwise it doesn’t seem to be learned fast enough empirically. This isn’t done for the shape tensors described by the following paragraph though. 22

For grids of non-constant shape, the [example, height, channel] and [example, width, channel] tensors are used to create distributions over possible contiguous rectangular slices of each grid of colors, as shown in Figure 11. Again, the channel dimension is mapped down to a size of 2 for input and output grids. For every grid, we have a vector of size [width] and a vector of size [height]. The log likelihood of every slice of the vector is taken to be the sum of the values within the slice, minus the values outside the slice. The log likelihoods for all the possible slices are then normalized to have total probability one, and the colors for every slice are given by the color logits defined in the previous paragraph. With the puzzle distribution now defined, we can now evaluate the log-likelihood of the observed target puzzle, to use as the reconstruction error. 10 E OTHER ARCHITECTURAL DETAILS E.1 RULES FOR LEGAL MULTITENSORS

- At least one non-example dimension must be included. Examples are not special for any reason not having to do with colors, directions, rows, and columns.
- If the width or height dimension is included, the example dimension should also be included. Positions are intrinsic to grids, which are indexed by the example dimension. Without a grid it doesn’t make as much sense to talk about positions. E.2 WEIGHT TYPING FOR REFLECTION/ROTATION SYMMETRY When applying a different linear layer to every tensor in a multitensor, we have a linear layer for tensors having a width but no height dimension, and another linear layer for tensors having a height but not width dimension. Whenever this is the case, we tie the weights together in order to preserve the whole network’s equivariance to diagonal reflections and 90 degree rotations, which swap the width and height dimensions. The softmax layer is not completely symmetrized because different indices of the output correspond to different combinations of dimensions to softmax over. Tying the weights properly would be a bit complicated and time consuming for the performance improvement we expect, so we did not do this. E.3 TRAINING/INITIALIZATION We train for 2000 iterations using Adam, with learning rate 0.01, β of 0.5, and β of 0.9. Weights 1 2 are essentially all initialized with Xavier normal initialization. F PREPROCESSING F.1 OUTPUT SHAPE DETERMINATION The raw data consists of grids of various shapes, while the neural network operates on grids of constant shape. Most of the preprocessing that we do is aimed towards this shape inconsistency problem. Before doing any training, we determine whether the given ARC-AGI puzzle follows three possible shape consistency rules:
- The outputs in a given ARC-AGI puzzle are always the same shape as corresponding inputs.
- All the inputs in the given ARC-AGI puzzle are the same shape. 10 There are multiple slices of the same shape that result in the correct puzzle to be decoded. We sum together the probabilities of getting any of the slices by applying a log sum exp to the log probabilities. But, we found empirically that training prematurely collapses onto one particular slice. So, we pre-multiply and post-divide the log probabilities by a coefficient when applying the log sum exp. The coefficient starts at 0.1 and increases exponentially to 1 over the first 100 iterations of training. We also pre-multiply the masks by the square of this coefficient as well, to ensure they are not able to strongly concentrate on one slice too early in training. 23
- All the outputs in the given ARC-AGI puzzle are the same shape. Based on rules 1 and 3, we try to predict the shape of held-out outputs, prioritizing rule 1 over rule

6. If either rule holds, we force the postprocessing step to only consider the predicted shape by overwriting the masks produced by the linear head layer. If neither rule holds, we make a temporary prediction of the largest width and height out of the grids in the given ARC-AGI puzzle, and we allow the mask to predict shape that is smaller than that. The largest width and height that is given or predicted, are used as the size of the multitenor's width and height dimensions. The predicted shapes are also used as masks when performing the multitenor communication, directional communication and directional cummax/shift layers. We did not apply masks for the other layers because of time constraints and because we do not believe it will provide for much of a performance improvement.
- 11 F.2 NUMBER OF COLORS We notice that in almost all ARC-AGI puzzles, colors that are not present in the puzzle are not present in the true answers. Hence, any color that does not appear in the puzzle are not given an index in the color dimension of the multitenor. In addition, black is treated as a special color that is never included in the multitenor, since it normally represents the background in many puzzles. When performing color classification, a tensor of zeros is appended to the color dimension after applying the linear head, to represent logits for the black color. G POSTPROCESSING Since the generated answer grid is stochastic from randomness in z, we save the answer grids throughout training, and roughly speaking, we choose the most frequently occurring one as our denoised final prediction. This is complicated by the variable shape grids present in some puzzles. Generally, when we sample answers from the network by taking the logits of the [example, color, height, width, channel] tensor and argmaxing over the color dimension, we find that the grids are noisy and will often have the wrong colors for several random pixels. We developed several methods for removing this noise:
7. Find the most commonly sampled answer.
8. Construct an exponential moving average of the output color logits before taking the softmax to produce probabilities. Also construct an exponential moving average of the masks.
9. Construct an exponential moving average of the output color probabilities after taking the softmax. Also construct an exponential moving average of the masks. When applying these techniques, we always take the slice of highest probability given the mask, and then we take the color of highest probability afterwards. We explored several different rules for when to select which method, and arrived at a combination of 1 and 2 with a few modifications: • At every iteration, count up the sampled answer, as well as the exponential moving average answer (decay=0.97). • If before 150 iterations of training, then downweight the answer by a factor of e−10. (Effectively, don't count the answer.) • If the answer is from the exponential moving average as opposed to the sample, then downweight the answer by a factor of e−4. 11 The two masks for the input and output are combined together to make one mask for use in these operations, since the channel dimension in these operations don't necessarily correspond to the input and output grids. 24

• Downweight the answer by a factor of e^{-10} * uncertainty, where uncertainty is the average (across pixels) negative log probability assigned to the top color of every pixel.

H EMPIRICALLY OBSERVED ABILITIES AND DISABILITIES OF COMPRESSARC

(b) CompressARC's solution to puzzle 28e73c20 (a) Puzzle 28e73c20 Figure 12: Puzzle 28e73c20, and CompressARC's solution to it. A short list of abilities that can be performed by CompressARC includes: • Assigning individual colors to individual procedures (see puzzle 0ca9ddb6) • Infilling (see puzzle 0dfd9992) • Cropping (see puzzle 1c786137) • Connecting dots with lines, including 45 degree diagonal lines (see puzzle 1f876c06) • Same color detection (see puzzle 1f876c06) • Identifying pixel adjacencies (see puzzle 42a50994) • Assigning individual colors to individual examples (see puzzle 3bd67248) • Identifying parts of a shape (see puzzle 025d127b) • Translation by short distances (see puzzle 025d127b) We believe these abilities to be individually endowed by select layers in the architecture, which we designed specifically for the purpose of conferring those abilities to CompressARC. A short list of abilities that cannot be performed by CompressARC includes: • Assigning two colors to each other (see puzzle 0d3d703e) • Repeating an operation in series many times (see puzzle 0a938d79) • Counting/numbers (see puzzle ce9e57f2) • Translation, rotation, reflections, rescaling, image duplication (see puzzles 0e206a2e, 5ad4f10b, and 2bcee788) • Detecting topological properties such as connectivity (see puzzle 7b6016b9) • Planning, simulating the behavior of an agent (see puzzle 2dd70a9a) • Long range extensions of patterns (see puzzle 28e73c20 above) 25

I BASELINES The U-Net baseline in Table 1 was created to observe the performance of a more standard approach when subject to the same constraints of CompressARC, namely the avoidance of any training before inference time, and the sole use of the test puzzle as training data during inference time. The training algorithm for the baseline consists of feeding each input grid into the U-Net and using the U-Net output to classify the pixel color of the output grid. Puzzles where the input grid and output grid did not match shape were skipped and assumed to receive a score of zero. The most common two output grids occurring in the second half of the 10000 steps of training were used as the two solution guesses. We did not change the width or height of the grids in order to fit the ARC-AGI grids into the U-Net. The U-Net's Batch Norm was replaced with a Group Norm, and the middle pooling/upsampling layers were skipped if the activation grids were too small to be pooled anymore. We experimented with applying a random augmentation transformation to the input grids and reversing the transformation on the output before computing the loss and/or ensembling predictions, but we discarded this idea due to worse performance on the training set (2.5% without augmentations, 1% with augmentations).

J PUZZLE	SOLVE	ACCURACY	TABLES
See Tables 4 and 5 for numerically reported puzzle solve accuracies on the whole dataset.			
Table 4: CompressARC's puzzle solve accuracy on the training set as a function of the number of steps of inference time learning it is given, for various numbers of allowed guesses (pass@n). The official benchmark is reported with 2 allowed guesses, which is why we report 20% on the evaluation set. Total training set solve time is reported for an NVIDIA RTX 4070 GPU by solving one puzzle at a time in a sequence.			
Training Iteration Time	Pass@1	Pass@2	Pass@5
	Pass@10	Pass@100	Pass@1000
	100	6h 1.00%	2.25% 3.50%
	-----	-----	-----
	200	13h 11.50%	14.25% 16.50%
	300	19h 18.50%	21.25% 23.50%
	400	26h 21.00%	25.00% 28.75%
	500	32h 23.00%	27.50% 31.50%
	750	49h 28.00%	30.50% 34.00%
	1000	65h 28.00%	31.75% 35.50%
	1250	81h 29.00%	32.25% 37.00%
	1500	97h 29.50%	33.00% 38.25%
	2000 130h	30.25%	34.75% 38.25%
K HOW	TO IMPROVE	OUR WORK	
At the time of release of CompressARC, there were several ideas which we thought of trying or attempted at some point, but didn't manage to get working for one reason or another. Some ideas we still believe in, but didn't use, are listed below.			
K.1 JOINT COMPRESSION VIA WEIGHT SHARING BETWEEN PUZZLES			
Template Algorithm 1 includes a hard-coded value of θ for every single puzzle. We might be able to further shorten the template program length by sharing a single θ between all the puzzles, enhancing the compression and creating more correct puzzle solutions. Algorithm 2 would have to be changed accordingly.			
To implement this, we would most likely explore strategies like:			
26			

Table 5: CompressARC's puzzle solve accuracy on the evaluation set, reported the same way as in Table 4. Training Iteration Time

100	7h 0.75%	1.25%	2.25%	2.50%	3.00%	3.00%
200	14h 5.00%	6.00%	7.00%	7.75%	12.00%	12.25%
300	21h 10.00%	10.75%	12.25%	13.25%	15.50%	16.25%
400	28h 11.75%	13.75%	16.00%	17.00%	19.75%	20.00%
500	34h 13.50%	15.00%	17.75%	19.25%	20.50%	21.50%
750	52h 15.50%	17.75%	19.75%	21.50%	22.75%	25.50%
1000	69h 16.75%	19.25%	21.75%	23.00%	26.00%	28.75%
1250	86h 17.00%	20.75%	23.00%	24.50%	28.25%	30.75%
1500	103h 18.25%	21.50%	24.25%	25.50%	29.50%	31.75%
2000	138h 18.50%	20.00%	24.25%	26.00%	31.25%	33.75%
• Using the same network weights for all puzzles, and training for puzzles in parallel. Each puzzle gets assigned some perturbation to the weights, that is constrained in some way, e.g., LORA (Hu et al., 2021).						

100	7h 0.75%	1.25%	2.25%	2.50%	3.00%	3.00%
• Learning a “puzzle embedding” for every puzzle that is a high-dimensional vector (more than 16						
dim, less than 256 dim), and learning a linear mapping from puzzle embedding to weights for our						
network. This mapping serves as a basic hypernetwork, i.e., a neural network that outputs weights						
for another neural network (Chauhan et al., 2024).						
Unfortunately, testing this would require changing CompressARC (Algorithm 3) to run all puzzles						
in parallel rather than one at a time in series. This would slow down the research iteration process,						
which is why we did not explore this option.						
K.2 CONVOLUTION-LIKE LAYERS FOR SHAPE COPYING TASKS						
This improvement is more ARC-AGI-specific and may have less to do with AGI in our view. Many						
ARC-AGI-1 puzzles can be seen to involve copying shapes from one place to another, and our						
network has no inductive biases for such an operation. An operation which is capable of copying						
shapes onto multiple locations is the convolution. With one grid storing the shape and another with						
pixels activated at locations to copy to, convolving the two grids will produce another grid with the						
shape copied to the designated locations.						
There are several issues with introducing a convolutional operation for the network to use. Ideally,						
we would read two grids via projection from the residual stream, convolve them, and write it back in						
via another projection, with norms in the right places and such. Ignoring the fact that the grid size						
changes during convolution (can be solved with two parallel networks using different grid sizes), the						
bigger problem is that convolution tends to amplify noise in the grids much more than the sparse						
signals, so their inductive bias is not good for shape copying. We can try to apply asoft max to one						
or both of the grids to reduce the noise (and to draw an interesting connection to attention), but we						
didn’t find any success.						
The last idea that we were tried before discarding the idea was to modify the functional form of the						
convolution:						
(cid:88)						
(f ∗ g)(x) = f(x − y)g(y)						
y						
to a tropical convolution (Fan et al., 2021), which we found to work well on toy puzzles, but not well						
enough for ARC-AGI-1 training puzzles (which is why we discarded this idea):						
(F ∗ g)(x) = max f(x − y) + g(y)						

y						
27						

Convolutions, when repeated with some grids flipped by 180 degrees, tend to create high activations at the center pixel, so sometimes it is important to zero out the center pixel to preserve the signal.

K.3 KLFLOOR FOR POSTERIOR COLLAPSE

We noticed during testing that crucial posterior tensors whose KL fell to zero during learning would never make a recovery and play their role in the encoding, just as in the phenomenon of mode collapse in variational autoencoders (vanden Oord et al., 2018). We believe that the KL divergence may upper bound the information content of the gradient training signal for parts of the network that process the encoded information. Thus, when a tensor z falls to zero KL, the network stops learning to use its encoded information, and the KL is no longer incentivized to recover. If we artificially hold the KL above zero for an extended period of training, then the network may learn to make use of the tensor’s information, incentivizing the KL to stay above zero when released again. We implemented a mechanism to keep the KL above a minimum threshold so that the network always learns to use that information, but we do not believe the network learns fast enough for this to be useful, as we have never seen a tensor recover before. Therefore, it might be useful to explore different ways to schedule this KL floor to start high and decay to zero, to allow learning when the KL is forced to be high, and to leave the KL unaffected later on in learning. This might cause training results to be more consistent across runs.

K.4 REGULARIZATION

In template Algorithm 1, we do not compress θ to reduce the number of bits it takes up. If we were to compress θ as well, this would produce an extra KL term to the loss in CompressARC (Algorithm 3), and the KL term would simplify to an L2 regularization on θ under certain reasonable limits. It is somewhat reckless for us to neglect compressing θ in our work due to the sheer number of bits θ contributes, and making this change may improve our results.

L ADDITIONAL DETAILS ABOUT THE ARC-AGI BENCHMARK

Hidden Rules: For every puzzle, there is a hidden rule that maps each input grid to each output grid. There are 400 training puzzles and they are easier to solve than the 400 evaluation puzzles. The training set is intended to help teach your system the following general themes which underlie the hidden rules in the evaluation set:

- **Objectness:** Objects persist and cannot appear or disappear without reason. Objects can interact or not depending on the circumstances.
- **Goal-directedness:** Objects can be animated or inanimate. Some objects are “agents”-they have intentions and they pursue goals.
- **Numbers & counting:** Objects can be counted or sorted by their shape, appearance, or movement using basic mathematics like addition, subtraction, and comparison.
- **Basic geometry & topology:** Objects can be shapes like rectangles, triangles, and circles which can be mirrored, rotated, translated, deformed, combined, repeated, etc. Differences in distances can be detected. The puzzles are redesigned so that humans can reasonably find the answer, but machines should have more difficulty. The average human can solve 76.2% of the training set, and a human expert can solve 98.5% (LeGriset et al., 2024).

Scoring: You are given some number of examples of input-to-output mappings, and you get two guesses to guess the output grid(s) for a given input grid, without being told the hidden rule. One guess consists of guessing the width and height of the output grid(s), as well as all the pixel colors within, and if each of these is correct, then that guess succeeds. If either guess succeeds, then you score 1 for that puzzle, else you score 0. Some puzzles have more than one input/output pair that you have to guess, in which case the score for that puzzle may be in between.

Scoring Environment: The competitions launched by the ARC Prize Foundation have been restricted to 12 hours of compute per solution submission, in a constrained environment with no internet access.

This is where a hidden semi-private evaluation set is used to score solutions. These scores were reported on the public evaluation set, which is of the same difficulty as the semi-private evaluation set, which we had no access to when we performed this work.

M ADDITIONAL CASE STUDIES

Below, we show two additional puzzles and a dissection of CompressARC’s solution to them.

M.1 CASE STUDY: BOUNDINGBOX

Puzzle 6d75e8bb from the training split, see Figure 13.

Figure 13: Bounding Box: Puzzle 6d75e8bb from the training split.

M.1.1 WATCHING THE NETWORK LEARN: BOUNDINGBOX Human Solution: We first realize that the input is red and black, and the output is also red and black, but some of the black pixels are replaced by light blue pixels. We see that the red shape remains unaffected. We notice that the light blue box surrounds the red shape, and finally that it is the smallest possible surrounding box that contains the red shape. At this point, we copy the input over to the answer grid, then we figure out the horizontal and vertical extent of the red shape, and color all of the non-red pixels within that extent as light blue.

CompressARC Solution: See Table 6.

M.1.2 SOLUTION ANALYSIS: BOUNDINGBOX

Figure 14 shows the amount of contained information in every tensor within z . All the tensors in z fall to zero information content during training, except for three tensors. From 600-1000 steps, we see the [example, height, width, channel] tensors suffer a massive drop in information content, with no change in the outputted answer. We believe it was being used to identify the light blue pixels in the input, but this information then got memorized by the non-linear portions of the network, using the [example, height, channel] and [example, width, channel] as positional encodings.

Figure 15 shows the average output of the decoding layer for these tensors to see what information is stored there.

M.2 CASE STUDY: CENTERCROSS

Puzzle 41e4d17 is part of the training split, see Figure 16a.

Table 6: CompressARC learning the solution for BoundingBox, over time. Learning

What is CompressARC doing?			Sampled solution guess	
steps				
	The average	of sampled	outputs shows	that
---	-----	-----	-----	---
light blue pixels in the input are generally pre-				
	served in the output.	However, black pixels in		
---	-----	-----	---	---
the input are haphazardly and randomly col-				
50	ored light blue and red.	CompressARC does		
---	-----	---	-----	---
not seem to know that the colored input/output				
pixels lie within some kind of bounding box,				
or that the bounding box is the same for the				

What is CompressARC doing?				Sampled solution guess
input and output grids.				
	The average	of sampled	outputs shows	red
---	-----	-----	-----	---
pixels confined to an imaginary rectangle sur-				
	rounding the light blue pixels.		CompressARC	
---	-----	---	-----	---
seem to have perceived that other examples				
use a common bounding box for the input and				
100	output pixels, but is not completely sure about			
---	-----	---	---	---
where the boundary lies and what colors go in-				
	side the box in the output.		Nevertheless, guess	
---	-----	---	-----	---
2 (the second most frequently sampled output)				
shows that the correct answer is already being				
sampled quite often now.				
The average of sampled output shows almost				
	all of the	pixels in the imaginary	bounding	
---	-----	-----	-----	-----
150	box to be	colored red.	CompressARC	has
	figured out	the answer,	and further	training
only refine the answer.				
30				

Figure 14: Breaking down the KL loss during training into contributions from each individual shaped tensor in the multitenzor. (a) [example, height, channel] tensor. The first principal component is 771 times stronger than the second principal component. A brighter pixel indicates a row with more light blue pixels. It indicates a column with more light blue pixels. It is unclear how CompressARC knows where the borders of the bounding box are. (c) [color, channel] tensor. This tensor serves to distinguish the roles of the two colors apart. Figure 15: Breaking down the loss components during training tells us where and how CompressARC prefers to store information relevant to solving a puzzle. Human Solution: We first notice that the input consists of blue “bubble” shapes (really they are just squares, but the fact that they’re blue reminds us of bubbles) on a light blue background and the output has the same. But in the output, there are now magenta rays emanating from the center of each bubble. We copy the input over to the answer grid, and then draw magenta rays starting from the center of each bubble out to the edge in every cardinal direction. At this point, we submit our answer and find that it is wrong, and we notice that in the given demonstrations, the blue bubble color is drawn on top of the magenta rays, and we have drawn the rays on top of the bubbles instead. So, we pick up the blue color and correct each point where a ray pierces a bubble, back to blue. 31

(b) CompressARC’s solution. (a) The puzzle. Figure 16: CenterCross: Puzzle 41e4d17e from the training split. CompressARC Solution: We don’t show CompressARC’s solution evolving over time because we think it is uninteresting; instead we will describe. We don’t see much change in CompressARC’s answer over time during learning. It starts by copying over the input grid, and at some point, magenta rows and columns start to appear, and they slowly settle on the correct positions. At no point does CompressARC mistakenly draw the rays on top of the bubbles; it has always had the order correct. M.2.1 SOLUTION ANALYSIS: CENTERCROSS Figure 17 shows another plot of the amount of information in every tensor in z. The only surviving tensors are the [color, channel] and [example, height, width, channel] tensors, which are interpreted in Figure 18. Figure 17: Breaking down the KL loss during training into contributions from each individual shaped tensor in the multitenzor. 32

(a) [example,	height,	width, channel]				
tensor.	The top principal	component	is 2496			
times stronger than the second principal compo-						
nent. This tensor codes for the centers of the						
bubbles. In the KL contribution plot, we can see						
that the information content of this tensor is de-						
creasing over time. Likely, CompressARC is in the						
process of eliminating the plus shaped representa-						
tion, and replacing it with a pixel instead, which						
			(b) [color, channel]	tensor.	This tensor just	
---	---	---	---	-----	-----	-----
takes fewer bits.						
			serves to distinguish	the individual	roles of the	
---	---	---	---	-----	-----	-----
colors in the puzzle.						
Figure 18: Breaking down the loss components during training tells us where and how CompressARC						
prefer to store information relevant to solving a puzzle.						
N LIST	OF MENTIONED	ARC-AGI-1	PUZZLES			
-----	-----	-----	---	-----	---	---
See Table 7 below.						
Puzzle025d127b			Puzzle0a938d79		Puzzle0ca9ddb6	
-----	---	---	-----	---	-----	---
33						
Puzzle0d3d703e	Puzzle0dfd9992	Puzzle0e206a2e				
Puzzle1c786137	Puzzle1f876c06	Puzzle28e73c20				
34						
Puzzle272f95fa	Puzzle2bcee788	Puzzle2dd70a9a				
Puzzle3bd67248	Puzzle41e4d17e	Puzzle42a50994				
35						

Puzzle 5ad4f10b Puzzle 6d75e8bb Puzzle 7b6016b9 Puzzle ce9e57f2 Table 7: List of Mentioned ARC-AGI=1 Puzzles. All these puzzles are part of the training split. O CODE Code for this project is provided at <https://github.com/iliao2345/CompressARC> 36

ARC-AGI SIN PREENTRENAMIENTO

Albert Gu
Carnegie Mellon University

Resumen

La sabiduría convencional en la era de los grandes modelos de lenguaje (LLM) sostiene que resolver rompecabezas visuales similares a pruebas de coeficiente intelectual, como los del benchmark ARC-AGI-1, requiere capacidades derivadas de un preentrenamiento masivo.

Como respuesta a esta idea, presentamos **CompressARC**, un modelo de únicamente **76 000 parámetros** que no utiliza ningún tipo de preentrenamiento y que resuelve el **20 %** de los rompecabezas del conjunto de evaluación minimizando la **Longitud Mínima de Descripción (Minimum Description Length, MDL)** del rompecabezas objetivo exclusivamente durante el tiempo de inferencia.

El principio MDL dota a CompressARC de unas capacidades de generalización extremadamente elevadas, poco habituales en aprendizaje profundo. Hasta donde sabemos, CompressARC es el único método basado en aprendizaje profundo para ARC-AGI en el que el entrenamiento se realiza únicamente sobre una sola muestra: el propio rompecabezas que se desea resolver durante la inferencia, eliminando previamente la información correspondiente a la solución final.

Además, CompressARC no se entrena utilizando el conjunto de entrenamiento proporcionado por ARC-AGI.

Bajo estas condiciones extremadamente limitadas en cuanto a disponibilidad de datos, normalmente no esperaríamos que pudiera resolverse ningún rompecabezas. Sin embargo, CompressARC consigue resolver una distribución diversa de problemas creativos de ARC-AGI, lo que sugiere que el principio de Longitud Mínima de Descripción constituye una vía alternativa y viable para producir inteligencia, además del enfoque convencional basado en preentrenamiento.

1. Introducción

El benchmark **ARC-AGI-1** consiste en una colección de rompecabezas de razonamiento visual abstracto diseñados para evaluar la capacidad de un sistema para adquirir rápidamente nuevas habilidades a partir de una cantidad mínima de datos de entrada (Chollet, 2019).

Los avances recientes en razonamiento basado en grandes modelos de lenguaje han demostrado capacidades impresionantes para adquirir nuevas habilidades. Sin embargo, estos sistemas siguen dependiendo de enormes cantidades de datos de preentrenamiento.

En este trabajo exploramos cuál es la cantidad mínima de información realmente necesaria para abordar ARC-AGI mediante la introducción de **CompressARC**, un método de resolución derivado del principio de **Longitud Mínima de Descripción (MDL)** (Rissanen, 1978).

CompressARC realiza todo su aprendizaje durante el tiempo de inferencia y alcanza una precisión del **20 %** sobre los rompecabezas del conjunto de evaluación de ARC-AGI-1 utilizando únicamente el propio rompecabezas que está resolviendo como fuente de datos.

La clave de la extraordinaria eficiencia de datos de CompressARC reside en formular el problema como un ejercicio de **code golf**: encontrar el programa autónomo más corto posible capaz de generar como salida el conjunto completo de datos de ARC-AGI, rellenando arbitrariamente las cuadrículas cuya solución aún se desconozca.

Siguiendo la navaja de Occam, cabe esperar que el programa más corto complete dichas cuadrículas de la forma más razonable posible, es decir, utilizando soluciones "correctas" que respeten las reglas implícitas del rompecabezas.

Un programa excesivamente simple podría limitarse a almacenar una cadena de texto con todos los datos del rompecabezas (junto con soluciones arbitrarias) e imprimirla como salida. Sin embargo, dicho programa sería demasiado largo, lo que implica que las soluciones producidas serían incorrectas.

Por otro lado, encontrar el programa absolutamente más corto requeriría enumerar exhaustivamente una enorme cantidad de programas candidatos, algo computacionalmente inviable (Solomonoff, 1964; Hutter, 2005).

CompressARC propone un nuevo equilibrio: sobreajusta una red neuronal específicamente al rompecabezas para comprimirlo dentro de las matrices de pesos de la red. Posteriormente, estos pesos pueden codificarse directamente dentro del programa en lugar de almacenar el rompecabezas completo, utilizándose después para reconstruir los rompecabezas memorizados junto con sus soluciones.

Contabilizando cuidadosamente el número de bits necesarios para representar dichos pesos, esta técnica transforma la búsqueda combinatoria del mejor programa en un problema de optimización diferenciable, permitiendo minimizar la longitud del programa en un tiempo razonable y generar predicciones de alta calidad para las soluciones.

Este planteamiento conserva varias propiedades especialmente atractivas del enfoque original basado en code golf, todas ellas novedosas dentro de las soluciones neuronales para ARC-AGI:

- **Sin preentrenamiento.** Dado que el rompecabezas objetivo ya está disponible, no existe una fase previa de entrenamiento.
- **Aprendizaje durante la inferencia.** La longitud del programa se minimiza exclusivamente durante la inferencia optimizando los pesos de la red con respecto al rompecabezas objetivo.
- **Necesidad mínima de datos.** Siguiendo la navaja de Occam, asumimos que el programa más corto posee una fuerte capacidad de generalización y utilizamos únicamente el propio rompecabezas, sin cargar información adicional en memoria.

A pesar de no utilizar nunca el conjunto de entrenamiento, no realizar ningún tipo de preentrenamiento y disponer únicamente de **76 000 parámetros**, CompressARC consigue generalizar hasta resolver el **20 %** de los rompecabezas del conjunto de evaluación y el **34,75 %** del conjunto de entrenamiento, un rendimiento que sería imposible para los métodos tradicionales de aprendizaje profundo bajo estas restricciones.

El buen rendimiento de CompressARC en este escenario sugiere que trasladar los principios de compresión de información a otros ámbitos con escasez de datos —como el descubrimiento de fármacos o el diseño de proteínas— podría permitir obtener capacidades significativamente superiores también en esas aplicaciones.

El resto del artículo está organizado de la siguiente forma:

- La Sección 2 introduce el benchmark ARC-AGI.
- La Sección 3 presenta la formulación del problema.
- La Sección 4 describe la arquitectura de CompressARC.
- La Sección 5 expone los resultados experimentales.
- La Sección 6 analiza las implicaciones del trabajo.

2. Antecedentes: el benchmark ARC-AGI

ARC-AGI-1 es un benchmark de inteligencia artificial diseñado para evaluar la capacidad de un sistema para adquirir nuevas habilidades a partir de un número muy reducido de ejemplos (Chollet, 2019).

Cada rompecabezas del benchmark está gobernado por una regla oculta distinta que el sistema debe descubrir y aplicar a una cuadrícula de entrada formada por colores para producir la cuadrícula objetivo correspondiente.

Estas reglas ocultas utilizan conceptos como:

- reconocimiento de objetos;
- comportamiento orientado a objetivos;
- números y conteo;
- geometría básica;
- topología.

Para ayudar al sistema a descubrir la regla, cada rompecabezas proporciona varios pares de ejemplos entrada–salida. No se facilita ninguna información adicional.

La Figura 1 muestra tres ejemplos de rompecabezas del conjunto de entrenamiento de ARC-AGI-1.

Figura 1. Tres ejemplos de rompecabezas de ARC-AGI-1.

- (a) **Regla oculta:** desplazar cada objeto un píxel hacia la derecha, excepto los bordes inferior y derecho del propio objeto.
- (b) **Regla oculta:** reducir el tamaño del objeto grande y asignarle el mismo color que los puntos dispersos.
- (c) **Regla oculta:** prolongar la línea verde hasta alcanzar la línea roja, girando cuando encuentra un obstáculo.

Mientras que las soluciones basadas en grandes modelos de lenguaje entrenados con datos a escala de Internet han alcanzado un **87,5 %** de precisión en este benchmark (Chollet, 2024), y las redes neuronales entrenadas únicamente con datos de ARC-AGI han logrado un **40,3 %** (Wang et al., 2025), CompressARC lleva el enfoque de minimización de datos hasta el extremo, entrenándose exclusivamente sobre el propio rompecabezas de prueba.

El Apéndice L proporciona información adicional sobre el benchmark ARC-AGI-1.

Asimismo, el Apéndice A contiene una revisión ampliada del trabajo relacionado, incluyendo otros enfoques desarrollados para resolver ARC-AGI.

A lo largo del resto del artículo nos referiremos generalmente a **ARC-AGI-1** simplemente como **ARC-AGI**.

3. Método

CompressARC intenta resolver el problema de comprimir los datos en un programa lo más corto posible, de manera que permita obtener las soluciones de los rompecabezas manteniendo, al mismo tiempo, una búsqueda del programa computacionalmente viable.

En este planteamiento, el programa debe ser completamente autónomo, no recibir ninguna entrada y producir como salida el conjunto completo de rompecabezas de ARC-AGI, incluyendo cualquier solución que haya sido rellenada.

Normalmente resulta inviable realizar una búsqueda algorítmica exhaustiva para encontrar el programa absolutamente más corto, ya que sería necesario explorar un número gigantesco de programas de longitud creciente hasta hallar uno cuya salida coincidiera exactamente con nuestros requisitos.

CompressARC hace que esta búsqueda sea mucho más manejable restringiéndola a un subespacio de programas cuidadosamente diseñado.

Nuestra estrategia general, que iremos desarrollando en las siguientes secciones, consiste en lo siguiente:

- Definimos un espacio de programas mediante un **programa plantilla** (Algoritmo 1), que contiene numerosos espacios vacíos destinados a almacenar valores codificados directamente ("semillas" o *seeds*). Estas semillas completan el programa para que pueda ejecutarse y generar los rompecabezas con sus soluciones.
- Definimos un algoritmo de búsqueda (Algoritmo 2) que explora múltiples semillas posibles para rellenar dichos espacios y selecciona aquellas que producen el programa más corto posible, imponiendo únicamente la restricción de que los rompecabezas generados coincidan con los rompecabezas conocidos, mientras que las soluciones permanecen libres. Dado que la mayor parte del tamaño del programa corresponde precisamente a las semillas, basta con contabilizar la longitud total de estas para aproximar la longitud del programa completo.
- Solamente una parte del Algoritmo 2 es realmente necesaria para producir las soluciones; el resto únicamente sirve para construir la representación comprimida del rompecabezas. Para resolver los rompecabezas conservamos únicamente esa parte esencial y aproximamos la búsqueda mediante descenso por gradiente (Algoritmo 3). Las aproximaciones diferenciables utilizadas para estimar la longitud de las distintas semillas son análogas a los diferentes componentes de la función de pérdida de un Autoencoder Variacional (VAE).

Formato de los datos

Cada rompecabezas se representa mediante un tensor con forma:

```
[n_example, width, height, 2]
```

que contiene el color de cada píxel de las dos cuadrículas correspondientes a cada par entrada–salida.

Aquí:

- n_example** representa el número total de pares entrada–salida del rompecabezas, incluyendo el par de prueba cuya salida es desconocida.

Las dimensiones indicadas en esta sección tienen únicamente un propósito explicativo; el formato real utilizado por la implementación se presenta posteriormente, en la Sección 4.

Un primer intento ingenuo de minimizar el tamaño del programa consistiría simplemente en escribir todos los rompecabezas dentro de una enorme cadena de texto e imprimirla.

Posteriormente podrían introducirse optimizaciones eliminando redundancias mediante bucles u otras construcciones del lenguaje.

Nuestro método adopta una estrategia distinta, descrita en las siguientes subsecciones.

3.1 Restricción del espacio de programas

Derivamos CompressARC seleccionando un subespacio concreto de programas definido por una plantilla (Algoritmo 1), que posteriormente se completa sustituyendo determinados valores codificados directamente dentro del programa.

Cada rompecabezas se genera de forma independiente, ejecutando las siguientes operaciones:

- Se genera aleatoriamente un tensor

```
z
```

con dimensiones

```
[n_example, n_colors, width, height, 2]
```

a partir de una distribución normal estándar.

Esta operación utiliza una semilla de generación cuya longitud será análoga al término KL de un VAE.

- El tensor se procesa mediante una red neuronal denominada

```
equivariant_NN
```

que produce un tensor de logits con dimensiones

```
[n_example, n_colors, width, height, 2]
```

- Finalmente se obtiene el rompecabezas muestreando colores de la distribución de probabilidad definida por dichos logits, produciendo un tensor de dimensiones

```
[n_example, width, height, 2]
```

Esperamos que los colores generados coincidan con los del rompecabezas real.

Este segundo proceso también utiliza una semilla, cuya longitud resulta análoga al término de reconstrucción de un VAE.

La red neuronal **equivariant_NN** posee una propiedad fundamental: es **equivariante** respecto a las transformaciones válidas de los rompecabezas ARC-AGI.

Esto significa que si:

- se permuta el orden de los ejemplos;
- se permutan los colores;
- se rota o refleja el tensor de entrada;

entonces la salida de la red sufrirá exactamente la misma transformación.

Algoritmo 1. Plantilla de un programa corto

La plantilla del programa realiza, para cada rompecabezas, las siguientes operaciones:

- Definir la arquitectura de la red neuronal equivariante.
- Insertar una semilla codificada (`seed_z`) correspondiente al rompecabezas.
- Generar el tensor de entrada aleatorio `z`.
- Insertar los pesos entrenados de la red neuronal.
- Ejecutar una propagación hacia delante (*forward pass*).
- Insertar una segunda semilla (`seed_error`).

- 7. Generar el rompecabezas completo.
- 8. Imprimir el rompecabezas.

Posteriormente el mismo bloque de código se repite para todos los rompecabezas del conjunto de datos.

La plantilla permite incorporar dos semillas pseudoaleatorias distintas para cada rompecabezas. Manipulando adecuadamente la segunda semilla puede garantizarse que el rompecabezas generado coincida exactamente con el rompecabezas verdadero del conjunto de datos, independientemente de cuál haya sido la primera semilla utilizada. Una vez garantizado esto, la longitud total del programa depende únicamente del número de bits necesarios para representar ambas semillas. Por tanto, buscar programas cortos equivale simplemente a encontrar semillas lo más cortas posible. Existen varias partes del programa que pueden modificarse para reducir esa longitud:

- las semillas de las líneas 3 y 7;
- los pesos de la red neuronal de la línea 5;
- la propia arquitectura definida en la línea 1.

Las dos primeras cuestiones se estudian en esta sección y la arquitectura se abordará en la Sección 4.

3.2 Optimización de las semillas

El Algoritmo 2 presenta un procedimiento para optimizar simultáneamente las semillas y los pesos del Algoritmo 1 con el objetivo de minimizar la longitud total del programa.

El procedimiento sigue tres pasos principales.

En primer lugar, modifica la semilla correspondiente a la generación del tensor z para que el muestreo aparente proceder de una distribución normal distinta, aprendida durante la optimización.

Posteriormente manipula la segunda semilla para garantizar que el rompecabezas generado coincida exactamente con el rompecabezas objetivo.

Finalmente aplica descenso por gradiente sobre:

- los parámetros de la distribución normal;
- los pesos de la red neuronal;

con el objetivo de minimizar la longitud total de las semillas.

La idea de manipular una semilla aleatoria para simular el muestreo desde otra distribución diferente presenta numerosas dificultades técnicas.

Bajo determinadas condiciones favorables esto puede realizarse mediante **Rejection Sampling**, que produce muestras de la distribución objetivo utilizando una semilla cuya longitud esperada depende del cociente máximo entre ambas distribuciones.

Posteriormente esta técnica puede mejorarse mediante **Relative Entropy Coding (REC)**, reduciendo la longitud esperada de la semilla hasta aproximarse a la divergencia de Kullback-Leibler (KL) entre ambas distribuciones.

No obstante, ejecutar REC requiere un tiempo exponencial respecto a dicha divergencia.

Para evitar este enorme coste computacional, CompressARC no ejecuta realmente REC.

En su lugar simula directamente las consecuencias que tendría dicho procedimiento.

Concretamente:

- el tensor z se genera directamente desde la distribución aprendida;
- la longitud de la primera semilla se aproxima mediante la divergencia KL entre la distribución aprendida y la distribución normal estándar;
- la longitud de la segunda semilla se aproxima mediante la entropía cruzada entre los logits producidos por la red y el rompecabezas conocido.

La demostración matemática de esta aproximación se presenta en el Apéndice B.

Algoritmo 2. Minimización de la Longitud de Descripción (MDL)

Entrada:

- Conjunto de datos ARC-AGI.

Procedimiento:

1. Definir la arquitectura de la red neuronal equivariante.
2. Para cada rompecabezas:
 - Inicializar aleatoriamente los pesos de la red.
 - Obtener las dimensiones del rompecabezas.
 - Inicializar una distribución normal parametrizada por media y covarianza diagonal.
3. Durante cada iteración:
 - Manipular la primera semilla para simular muestras procedentes de la distribución aprendida.
 - Generar el tensor latente z .
 - Ejecutar la red neuronal.
 - Manipular la segunda semilla para obtener exactamente el rompecabezas deseado.
 - Generar el rompecabezas completo.
 - Calcular la longitud total de ambas semillas.
 - Actualizar los parámetros mediante el optimizador Adam.
4. Insertar en el programa plantilla las semillas y los pesos obtenidos.
5. Repetir el procedimiento para todos los rompecabezas.
6. Devolver finalmente el programa comprimido.

Algoritmo 3. CompressARC

CompressARC es una versión simplificada del Algoritmo 2.

En lugar de realizar la manipulación explícita de semillas mediante REC, sustituye dicho procedimiento por una aproximación diferenciable.

El algoritmo funciona de la siguiente manera:

1. Inicializar la arquitectura.
2. Para cada rompecabezas:
 - Inicializar los pesos de la red.

- Inicializar la distribución latente.

3. Repetir durante el entrenamiento:

- Muestrear directamente

$$\mathbf{z} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$$

- Ejecutar la red neuronal.
- Calcular la pérdida

$$\text{KL}(\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma}) \parallel \mathcal{N}(\mathbf{0}, \mathbf{I})) + \text{CrossEntropy}(\text{logits}, \mathbf{P})$$

que aproxima la longitud del programa.

- Actualizar los parámetros mediante Adam.

4. Una vez finalizado el entrenamiento:

- Muestrear la solución producida por la red.
- Añadir dicha solución al conjunto de rompecabezas resueltos.

5. Devolver todas las soluciones obtenidas.

Gracias a esta formulación, CompressARC puede comprimir automáticamente el conjunto de datos ARC-AGI refinando sucesivamente el programa plantilla hasta producir las soluciones.

El único componente que aún queda por definir es la arquitectura concreta de la red neuronal utilizada dentro de dicha plantilla, cuestión que constituye el contenido de la siguiente sección.

4. Arquitectura

La idea central detrás del diseño de la arquitectura de la red neuronal consiste en maximizar la probabilidad de generar los rompecabezas de ARC-AGI mediante muestreo. Al conseguirlo, disminuye la longitud de las semillas y, en consecuencia, la longitud total del programa descrito en el Algoritmo 1.

De acuerdo con la estructura de dicho algoritmo, esto implica que la red neuronal debe poseer **buenos sesgos inductivos (inductive biases)** para transformar ruido aleatorio en rompecabezas que parezcan plausibles dentro del dominio de ARC-AGI.

Es importante destacar que los rompecabezas del conjunto de entrenamiento **no intervienen directamente** en nuestro método. Su única utilidad consiste en ayudar al diseño manual de estos sesgos inductivos dentro de la arquitectura.

Equivarianza

Un rompecabezas de ARC-AGI sigue siendo esencialmente el mismo independientemente de:

- el orden en que aparezcan los ejemplos;
- el orden asignado a los colores;
- la orientación de la cuadrícula;
- las reflexiones horizontales o verticales.

Por tanto, deseamos que la red neuronal asigne inicialmente la **misma probabilidad** a todas estas variantes.

Para conseguirlo, diseñamos una arquitectura **equivariante** respecto a:

- permutaciones de los ejemplos;
- permutaciones de colores;
- rotaciones;
- reflexiones.

La equivarianza garantiza que, si la entrada experimenta cualquiera de estas transformaciones, la salida de la red sufrirá exactamente la misma transformación.

En consecuencia, cualquier asimetría presente en un rompecabezas concreto no está codificada en la arquitectura de la red, sino que debe introducirse modificando la semilla aleatoria utilizada para generar el tensor latente **z**, inclinando así la distribución hacia una solución concreta.

Estructura general de la red neuronal

La Figura 3 muestra la estructura principal de la arquitectura utilizada por CompressARC.

La red trabaja sobre una representación denominada **multitensor**, que será descrita con detalle posteriormente.

La arquitectura está formada por tres grandes bloques:

1. **Capa de decodificación**, que desempeña un papel similar al de una matriz de embeddings.
2. **Núcleo principal (core)**, basado en una conexión residual.
3. **Capa lineal de salida**, aplicada sobre la dimensión correspondiente a los canales.

Dentro del núcleo residual, distintas proyecciones lineales leen información desde el estado residual y la envían hacia operaciones especialmente diseñadas para ARC-AGI.

Una vez procesada, la información vuelve a incorporarse al estado residual mediante otra proyección lineal.

A lo largo de toda la red aparecen también operaciones de normalización.

El bloque completo del núcleo se repite **cuatro veces**.

Comparación con Transformers

La filosofía general recuerda a la arquitectura Transformer.

En ambos casos existe:

- un flujo residual donde se almacena la información;
- operaciones especializadas que leen y escriben sobre dicho flujo;
- una capa lineal final encargada de producir la salida.

La diferencia fundamental es que, en lugar de utilizar mecanismos de atención (*attention*) sobre secuencias de tokens, CompressARC incorpora operaciones específicamente diseñadas para manipular rompecabezas bidimensionales.

Operaciones especializadas

Las principales operaciones implementadas por la arquitectura son:

1. Suma y difusión (*broadcast*)

Permite sumar un tensor a lo largo de uno o varios ejes y posteriormente propagar el resultado a dimensiones superiores.

Esta operación facilita compartir información global entre distintas partes del rompecabezas.

2. Softmax

La función Softmax puede aplicarse sobre uno o varios ejes del multitensor.

Esto permite construir distribuciones de probabilidad adaptadas a diferentes dimensiones del problema.

3. Máximo acumulado (*Cumulative Max*)

Calcula máximos acumulados a lo largo de una dimensión geométrica.

Esta operación rompe deliberadamente determinadas simetrías geométricas, proporcionando capacidades espaciales específicas.

4. Desplazamiento (*Shift*)

Desplaza el contenido un píxel a lo largo de una dimensión espacial.

Este tipo de operación resulta especialmente útil para resolver numerosos rompecabezas de ARC-AGI donde aparecen traslaciones de objetos.

5. No linealidad elemento a elemento

Se aplica una función no lineal independiente sobre cada componente del tensor.

6. Normalización

Se realiza una normalización sobre la dimensión de canales.

Además de estas operaciones, la arquitectura incorpora otra operación adicional descrita en el Apéndice D.5.

Parámetros del modelo

Un aspecto interesante de la arquitectura es que las operaciones descritas anteriormente **no poseen parámetros propios**.

Su comportamiento está completamente determinado por las proyecciones lineales que leen y escriben sobre el estado residual.

Estas proyecciones trabajan únicamente sobre la dimensión de canales.

Para reducir el coste computacional:

- algunas partes del flujo residual utilizan **16 canales**;
- otras utilizan únicamente **8 canales**.

Como consecuencia, el número total de parámetros del modelo es de aproximadamente **76 000**, una cifra extraordinariamente pequeña para un sistema capaz de resolver problemas de ARC-AGI.

El formato Multitensor

Aunque anteriormente describimos los datos mediante un único tensor, la implementación real utiliza una estructura mucho más rica denominada **multitensor**.

En lugar de trabajar con un único tensor de dimensiones

```
[n_example,
 n_colors,
 width,
 height,
 channel]
```

la red mantiene simultáneamente numerosos tensores distintos.

Cada uno contiene únicamente un subconjunto de dichas dimensiones.

Por ejemplo, pueden existir tensores con forma:

```
[n_colors,
 width,
 channel]
```

o

```
[n_example,
 height,
 channel]
```

El tensor de entrada **z**, los logits producidos por la red y todas las activaciones internas adoptan este formato multitensor.

En términos generales, existe un tensor para prácticamente cada subconjunto posible de dimensiones.

Este diseño permite incorporar importantes sesgos inductivos.

Por ejemplo, una correspondencia entre columnas de la cuadrícula y determinados colores puede almacenarse naturalmente mediante un tensor con dimensiones:

```
[color,
 width,
 channel]
```

Los autores remiten al Apéndice C para una descripción completa del funcionamiento interno de los multitensores.

5. Resultados

Para cada rompecabezas se ejecutaron **2 000 iteraciones de entrenamiento durante la inferencia**, con un tiempo aproximado de **20 minutos por rompecabezas**.

Bajo este presupuesto computacional, CompressARC consiguió resolver correctamente:

- **20 %** de los rompecabezas del conjunto de evaluación.
- **34,75 %** de los rompecabezas del conjunto de entrenamiento.

La Figura 4 muestra cómo aumenta la precisión conforme se permite un mayor número de iteraciones de entrenamiento durante la inferencia.

Las Tablas 4 y 5 del Apéndice contienen los resultados numéricos completos, incluyendo los tiempos de ejecución.

Tabla 1. Comparación con otros métodos

Los autores comparan CompressARC con distintos enfoques existentes para resolver ARC-AGI.

Cada método dispone de dos intentos de solución por rompecabezas.

Una solución únicamente se considera correcta cuando:

- la forma de la cuadrícula coincide exactamente;
- todos los colores de todos los píxeles son correctos.

Como referencia adicional se incluye un modelo U-Net entrenado únicamente durante la inferencia utilizando los ejemplos del propio rompecabezas de prueba, con el fin de mantener las mismas condiciones que CompressARC.

Los resultados son los siguientes:

Método	Datos utilizados	Red neuronal	Precisión
Adivinanza aleatoria	Ninguno	No	0 %
Búsqueda exhaustiva de reglas	Ninguno	No	40 % (evaluación privada)
U-Net	Solo rompecabezas objetivo	Sí	0,75 %
CompressARC	Solo rompecabezas objetivo	Sí	20 %
HRM (ablación)	Rompecabezas de prueba	Sí	31 %
HRM	Entrenamiento + prueba	Sí	40,3 %
OpenAI o3 High	Datos a escala de Internet	Sí	87,5 %

5.1 ¿Qué rompecabezas puede y no puede resolver CompressARC?

CompressARC intenta explotar al máximo las capacidades que posee hasta encontrarse con alguna limitación de su arquitectura.

Los autores ilustran esta situación mediante el rompecabezas **28e73c20** del conjunto de entrenamiento.

En este problema es necesario prolongar un patrón desde el borde hacia el centro de la cuadrícula.

Las capas de la red permiten extender patrones correctamente únicamente a distancias relativamente cortas.

En consecuencia, CompressARC prolonga correctamente el patrón durante varios píxeles, pero cuando debe alcanzar el centro de la cuadrícula ya no dispone de suficiente capacidad para continuar de forma fiable y termina realizando una estimación.

El Apéndice H recoge de forma sistemática las capacidades que los autores han observado experimentalmente que CompressARC posee y aquellas que todavía no consigue realizar.

5.2 Estudio de caso: "Color the Boxes"

El siguiente rompecabezas analizado por los autores recibe el nombre de **Color the Boxes**.

En él, varias cajas deben colorearse dependiendo de la posición que ocupan dentro de la cuadrícula.

5.2 Estudio de caso: "Color the Boxes"

El rompecabezas mostrado en la Figura 5 recibe el nombre de **Color the Boxes**.

En este problema, cada caja debe colorearse según el lado de la cuadrícula en el que se encuentra.

Solución humana

Una persona resolvería este rompecabezas aproximadamente de la siguiente manera.

En primer lugar, observa que la cuadrícula de entrada está dividida en varias cajas y que esas mismas cajas siguen existiendo en la salida; la única diferencia es que ahora aparecen coloreadas.

El siguiente paso consiste en descubrir qué color corresponde a cada caja.

Al analizar los ejemplos proporcionados se observan varios patrones:

- Las esquinas siempre son negras.
- La caja central siempre es de color magenta.
- El color de las cajas situadas en los laterales depende de la dirección en la que se encuentran respecto al centro:
 - arriba → rojo;
 - abajo → azul;
 - derecha → verde;
 - izquierda → amarillo.

Una vez identificada esta regla, basta con copiar la cuadrícula de entrada y colorear cada caja según dichas correspondencias.

Solución obtenida por CompressARC

La Tabla 2 muestra cómo evoluciona el comportamiento de CompressARC durante el entrenamiento realizado en tiempo de inferencia.

Una vez finalizado el aprendizaje, los autores analizan la distribución latente **z** aprendida y descubren que esta codifica esencialmente dos tipos de información:

- una tabla que relaciona colores con direcciones;
- las posiciones de las líneas divisorias entre filas y columnas.

Durante el entrenamiento, el error de reconstrucción disminuye extremadamente rápido.

Aunque posteriormente permanece bajo durante la mayor parte del proceso, ocasionalmente aparecen picos de error.

Cuando esto ocurre, la divergencia KL correspondiente al espacio latente aumenta temporalmente, tal y como puede observarse en la Figura 6(a).

5.2.1 Análisis de la solución

Los autores estudian las representaciones almacenadas dentro del espacio latente **z** para comprender cómo aprende CompressARC a resolver el rompecabezas.

Recordemos que **z** no es un único tensor, sino un **multitensor**.

Cada tensor que forma parte del multitensor contribuye de manera independiente al valor total de la divergencia KL.

Analizando la contribución individual de cada tensor (Figura 6(b)) es posible identificar cuáles contienen realmente información útil para representar el rompecabezas.

Durante el entrenamiento ocurre algo especialmente interesante.

Prácticamente todos los tensores terminan reduciendo su contenido informativo hasta cero.

Sin embargo, **cuatro tensores** conservan información significativa.

En algunas repeticiones del experimento incluso uno de estos cuatro tensores llega a perder toda su información.

Cuando eso sucede, CompressARC normalmente ya no consigue recuperar la solución correcta.

La ejecución mostrada por los autores corresponde a un caso afortunado en el que uno de dichos tensores casi pierde toda su información, pero vuelve a recuperarla aproximadamente alrededor de la iteración 200.

Precisamente a partir de ese momento comienzan a aparecer muestras generadas con los colores correctos en las cajas adecuadas.

Qué representa cada tensor

Los autores inspeccionan la salida media de la capa de decodificación asociada a cada tensor del espacio latente.

Cada tensor contiene vectores de dimensión igual al número de canales.

Aplicando **Análisis de Componentes Principales (PCA)** sobre dichos vectores puede determinarse cuántos conceptos distintos codifica cada tensor.

Los resultados muestran lo siguiente.

Tensor [ejemplo, altura, canal] y [ejemplo, anchura, canal]

El componente principal dominante identifica automáticamente las posiciones de las filas y columnas que delimitan las cajas.

Concretamente:

- los píxeles con mayor intensidad indican la posición de las líneas horizontales;
- los píxeles con menor intensidad indican la posición de las líneas verticales.

De esta manera la red aprende la estructura geométrica del rompecabezas.

Tensor [dirección, color, canal]

El componente principal dominante de este tensor representa directamente la correspondencia entre direcciones y colores.

Los autores encuentran la siguiente asociación:

- azul ↔ arriba;
- verde ↔ izquierda;
- rojo ↔ abajo;
- amarillo ↔ derecha.

Este tensor indica qué color debe utilizarse para cada dirección correspondiente a las cajas situadas alrededor del centro.

La primera componente principal es **829 veces** más importante que la segunda, indicando que prácticamente toda la información relevante se concentra en una única dimensión.

Interpretación

Este análisis muestra que CompressARC no memoriza simplemente la solución final.

En su lugar aprende representaciones internas compactas que describen:

- la geometría del rompecabezas;
- la relación entre direcciones y colores;
- la posición de los separadores de filas y columnas.

Posteriormente reconstruye la solución utilizando dichas representaciones comprimidas.

6. Discusión

La dependencia del aprendizaje profundo moderno respecto a enormes cantidades de datos de alta calidad limita seriamente su aplicación a problemas donde los datos disponibles son escasos.

Este problema resulta especialmente evidente en ARC-AGI.

En este benchmark dominan actualmente los grandes modelos de lenguaje entrenados sobre conjuntos de datos especialmente ampliados, seleccionados y cuidadosamente preparados.

En este contexto presentamos CompressARC, un sistema que no solo prescinde completamente de los datos de entrenamiento, sino que elimina incluso la fase completa de preentrenamiento.

Intuitivamente cabría esperar que un enfoque así fuera incapaz de resolver absolutamente ningún rompecabezas.

Sin embargo, al aplicar el principio de Longitud Mínima de Descripción (MDL) directamente sobre el rompecabezas objetivo durante la inferencia, CompressARC consigue resolver una fracción sorprendentemente elevada de los problemas de ARC-AGI-1.

Fundamentos teóricos

La base teórica de CompressARC consiste en minimizar la longitud de una descripción algorítmica del rompecabezas objetivo.

Las estrategias tradicionales basadas en MDL apenas se han utilizado debido a que el espacio de búsqueda de programas posibles resulta prácticamente inabordable.

CompressARC evita este problema restringiendo la búsqueda a un espacio formado por redes neuronales optimizable mediante descenso por gradiente.

Aunque la arquitectura utilizada ha sido cuidadosamente diseñada a mano, su extraordinaria capacidad para generalizar utilizando únicamente dos pares entrada–salida sitúa a CompressARC en un régimen de generalización completamente distinto al observado habitualmente en ARC-AGI.

Trabajo futuro

Los autores consideran que la principal línea de mejora consiste en aumentar la eficiencia computacional.

La arquitectura utiliza numerosas operaciones personalizadas.

Implementaciones optimizadas mediante:

- compilación JIT;
- kernels CUDA fusionados;

permitirían acelerar significativamente cada iteración de entrenamiento.

Estas mejoras resultarían especialmente beneficiosas para rompecabezas de gran tamaño, ya que el tiempo de ejecución de la arquitectura aumenta con el número de píxeles presentes en la cuadrícula.

Conclusión

Los autores cuestionan la hipótesis ampliamente aceptada de que la inteligencia debe surgir necesariamente del preentrenamiento masivo y del uso de enormes volúmenes de datos.

En su lugar muestran que una utilización inteligente del principio de Longitud Mínima de Descripción y de las técnicas de compresión de información puede producir capacidades sorprendentemente elevadas.

CompressARC constituye una prueba de concepto que demuestra que los marcos modernos de aprendizaje profundo pueden combinarse con MDL para ofrecer una vía alternativa —y complementaria— hacia el desarrollo de una Inteligencia Artificial General (AGI).

Apéndice A. Trabajo relacionado

En esta sección revisamos los trabajos más relevantes relacionados con CompressARC.

Nuestro método se sitúa en la intersección de varias áreas de investigación:

- ARC-AGI;

- aprendizaje en tiempo de inferencia (*test-time learning*);
- principio de Longitud Mínima de Descripción (MDL);
- codificación mediante entropía relativa (Relative Entropy Coding, REC);
- aprendizaje profundo con escasez extrema de datos.

A.1 ARC-AGI

Desde la publicación del benchmark ARC-AGI por Chollet (2019), se han propuesto numerosos enfoques para resolver este conjunto de problemas.

En términos generales, estos métodos pueden agruparse en tres grandes categorías.

Sistemas simbólicos

Los primeros trabajos se centraron principalmente en sistemas simbólicos.

Estos métodos intentan descubrir explícitamente una secuencia de transformaciones capaz de convertir las cuadrículas de entrada en las cuadrículas de salida.

Entre las técnicas utilizadas se encuentran:

- motores de búsqueda sobre programas;
- gramáticas de transformaciones;
- sistemas basados en reglas;
- lenguajes específicos de dominio (DSL).

Una ventaja importante de estos métodos consiste en que pueden generalizar con muy pocos ejemplos.

Sin embargo, el espacio de búsqueda suele crecer muy rápidamente, haciendo difícil resolver rompecabezas complejos.

Métodos basados en aprendizaje profundo

Otra línea de investigación consiste en entrenar redes neuronales utilizando el conjunto de entrenamiento de ARC-AGI.

En este caso la red intenta aprender regularidades comunes presentes en numerosos rompecabezas diferentes.

Diversos trabajos han empleado arquitecturas como:

- CNN;
- Transformers;
- Graph Neural Networks;
- modelos híbridos.

Estos enfoques suelen obtener mejores resultados cuando existe suficiente cantidad de datos de entrenamiento.

Sin embargo, su capacidad de generalización disminuye considerablemente cuando aparecen reglas completamente nuevas.

Grandes modelos de lenguaje

Los avances recientes en grandes modelos de lenguaje han permitido obtener resultados muy superiores.

Estos sistemas utilizan:

- enormes cantidades de datos de preentrenamiento;
- aprendizaje mediante instrucciones;
- cadenas de razonamiento (*Chain of Thought*);
- búsqueda durante la inferencia.

Aunque alcanzan niveles de precisión muy elevados, resulta difícil determinar hasta qué punto dichas capacidades proceden realmente del razonamiento o del conocimiento adquirido durante el preentrenamiento.

CompressARC explora precisamente el extremo opuesto.

Nuestro sistema elimina completamente el preentrenamiento y utiliza únicamente el propio rompecabezas que está resolviendo.

A.2 Aprendizaje durante la inferencia

El aprendizaje durante la inferencia (*test-time learning*) ha recibido una atención creciente durante los últimos años.

En estos métodos el modelo continúa adaptando sus parámetros después de haber finalizado el entrenamiento convencional.

Las aplicaciones incluyen:

- adaptación a cambios de distribución (*distribution shift*);
- personalización;
- metaaprendizaje;
- aprendizaje continuo.

CompressARC lleva esta idea mucho más lejos.

En nuestro caso:

- no existe entrenamiento previo;
- todos los parámetros comienzan con valores aleatorios;
- el aprendizaje completo ocurre exclusivamente durante la inferencia.

Hasta donde sabemos, no existen trabajos anteriores que entrenen una red neuronal desde cero utilizando únicamente un único rompecabezas de ARC-AGI y aun así consigan una capacidad apreciable de generalización.

A.3 Longitud Mínima de Descripción (MDL)

El principio de Longitud Mínima de Descripción fue introducido por Rissanen (1978).

La idea fundamental consiste en que la mejor explicación para un conjunto de datos es aquella que permite describirlos utilizando el menor número posible de bits.

Desde entonces MDL ha sido ampliamente utilizado en áreas como:

- selección de modelos;
- inferencia estadística;
- compresión de datos;
- aprendizaje automático.

CompressARC aplica este principio de una forma diferente.

En lugar de seleccionar entre varios modelos previamente entrenados, optimiza directamente un programa capaz de reconstruir el rompecabezas completo.

El objetivo de entrenamiento coincide exactamente con la longitud de dicho programa.

A.4 Codificación mediante Entropía Relativa (REC)

Relative Entropy Coding (REC) es una técnica desarrollada para generar muestras procedentes de una distribución objetivo utilizando una fuente de aleatoriedad diferente.

La longitud esperada de la información adicional necesaria depende de la divergencia de Kullback-Leibler entre ambas distribuciones.

REC proporciona una interpretación especialmente elegante del término KL presente en los Autoencoders Variacionales.

Sin embargo, su coste computacional aumenta exponencialmente con la divergencia.

Por este motivo CompressARC no ejecuta literalmente REC.

En su lugar optimiza directamente una función de pérdida equivalente que aproxima el número esperado de bits requerido por dicho procedimiento.

Una demostración formal de esta equivalencia aparece en el Apéndice B.

A.5 Aprendizaje con cantidades extremadamente reducidas de datos

La mayoría de los métodos modernos de aprendizaje profundo requieren grandes cantidades de ejemplos etiquetados.

Se han propuesto numerosas estrategias para reducir esta dependencia.

Entre ellas destacan:

- aprendizaje auto-supervisado;
- metaaprendizaje;
- aprendizaje por transferencia;
- aumento de datos (*data augmentation*);
- modelos fundacionales preentrenados.

CompressARC adopta una estrategia completamente distinta.

No intenta reutilizar conocimientos obtenidos previamente.

En su lugar utiliza el principio de compresión para inducir directamente la estructura presente en el rompecabezas objetivo.

Esta filosofía resulta especialmente adecuada para dominios donde apenas existen datos disponibles y donde las reglas cambian continuamente entre instancias.

Resumen del trabajo relacionado

En conjunto, los trabajos anteriores muestran dos tendencias claramente diferenciadas.

Por un lado, los sistemas simbólicos ofrecen una excelente eficiencia en cuanto al uso de datos, pero presentan dificultades para escalar.

Por otro, los modelos neuronales modernos escalan con gran facilidad, aunque normalmente necesitan enormes conjuntos de entrenamiento.

CompressARC intenta combinar las ventajas de ambos enfoques.

Utiliza una arquitectura neuronal optimizable mediante descenso por gradiente, pero guiando el aprendizaje mediante el principio clásico de Longitud Mínima de Descripción en lugar de depender del preentrenamiento masivo.

Apéndice B. Derivación de la función objetivo

En este apéndice demostramos que la función de pérdida utilizada por CompressARC constituye una aproximación diferenciable de la longitud del programa descrito en el Algoritmo 1.

La demostración se basa en los resultados conocidos sobre **Relative Entropy Coding (REC)**.

B.1 Longitud esperada de la primera semilla

Recordemos que el tensor latente

$$z$$

se genera originalmente a partir de una distribución normal estándar

$$\mathcal{N}(0, \mathbf{I}) .$$

Sin embargo, durante la optimización deseamos generar muestras procedentes de otra distribución

$$q(z) .$$

Una posibilidad consiste en utilizar **Relative Entropy Coding**.

REC permite transformar una secuencia de bits aleatorios obtenidos de la distribución base en muestras distribuidas según la distribución objetivo.

El resultado fundamental establece que la longitud esperada de la información adicional necesaria es aproximadamente

$$\text{KL}(q(z) \parallel \mathcal{N}(0, \mathbf{I})) .$$

Por tanto, el número esperado de bits requerido para codificar la primera semilla coincide con la divergencia KL entre ambas distribuciones.

Este término aparece exactamente igual que en un Autoencoder Variacional.

CompressARC utiliza directamente esta cantidad como aproximación diferenciable de la longitud de dicha semilla.

B.2 Longitud esperada de la segunda semilla

Una vez generado el tensor latente

$$z,$$

la red neuronal produce una distribución categórica sobre los posibles colores de cada píxel.

Denotemos por

$$p(x|z)$$

la probabilidad asignada al rompecabezas correcto.

Si utilizáramos nuevamente REC para forzar que el resultado generado coincidiera exactamente con el rompecabezas objetivo, la longitud esperada de la segunda semilla sería

$$-\log p(x|z).$$

Cuando la distribución es categórica, esta cantidad coincide exactamente con la pérdida de entropía cruzada (*cross-entropy*).

En consecuencia, el término de reconstrucción utilizado por CompressARC constituye una aproximación directa del número esperado de bits necesarios para codificar dicha semilla.

B.3 Longitud total del programa

Puesto que ambas semillas son independientes, la longitud total esperada del programa puede aproximarse simplemente sumando ambas contribuciones:

$$\begin{aligned} \text{Longitud} \approx & \\ & \text{KL}(q(z) \parallel N(0, I)) \\ & + \\ & \text{CrossEntropy}(x, \text{logits}). \end{aligned}$$

Esta expresión coincide exactamente con la función de pérdida optimizada durante el entrenamiento.

De este modo, minimizar la pérdida equivale a minimizar una aproximación diferenciable de la longitud del programa.

B.4 Relación con los Autoencoders Variacionales

La similitud entre CompressARC y un Autoencoder Variacional resulta evidente.

Ambos optimizan una función objetivo formada por dos términos:

- una divergencia KL;
- un término de reconstrucción.

Sin embargo, la interpretación es completamente distinta.

En un VAE convencional:

- la divergencia KL actúa como regularizador;
- el término de reconstrucción mide el error de reconstrucción de los datos.

En CompressARC ambos términos representan directamente cantidades de información:

- la longitud de la primera semilla;
- la longitud de la segunda semilla.

Por tanto, la función objetivo no se introduce como una heurística de regularización, sino como una estimación explícita de la longitud del programa buscado.

B.5 Aproximación diferenciable

La ejecución literal del Algoritmo 2 exigiría implementar Relative Entropy Coding durante cada iteración de entrenamiento.

Este procedimiento tendría un coste computacional prohibitivo.

En lugar de ello, CompressARC sustituye REC por una aproximación diferenciable.

Durante el entrenamiento:

1. se generan directamente muestras de la distribución aprendida;
2. se añade el término KL como estimación del coste de modificar la primera semilla;
3. se utiliza la entropía cruzada como estimación del coste de modificar la segunda semilla.

Esta sustitución permite aplicar descenso por gradiente convencional.

Los experimentos muestran que dicha aproximación resulta suficientemente precisa para optimizar la longitud del programa de forma efectiva.

B.6 Interpretación desde la teoría de la información

Desde un punto de vista de teoría de la información, CompressARC intenta responder a la siguiente pregunta:

¿Cuál es el menor número posible de bits necesarios para describir este rompecabezas concreto?

La respuesta se obtiene repartiendo la información entre tres componentes:

- la arquitectura de la red neuronal;
- los pesos aprendidos;
- las dos semillas pseudoaleatorias.

La arquitectura permanece fija.

Los pesos se optimizan para capturar regularidades compartidas dentro del propio rompecabezas.

Las semillas únicamente almacenan la información que no puede describirse mediante dichas regularidades.

Como consecuencia, cuanto mejor consiga la red explicar la estructura del rompecabezas, menor será la información residual que deberán almacenar las semillas.

Esto constituye exactamente el principio de Longitud Mínima de Descripción.

B.7 Consecuencias prácticas

Esta formulación presenta varias propiedades interesantes.

En primer lugar, la función objetivo posee una interpretación probabilística rigurosa.

No se trata simplemente de una pérdida diseñada empíricamente, sino de una aproximación explícita al número esperado de bits necesarios para representar el programa.

En segundo lugar, el entrenamiento deja de depender de conjuntos de datos externos.

Toda la información utilizada procede exclusivamente del rompecabezas que se desea resolver.

Finalmente, el enfoque conecta tres áreas que normalmente se estudian por separado:

- compresión de información;
- Autoencoders Variacionales;
- aprendizaje durante la inferencia.

CompressARC muestra que estas tres perspectivas pueden integrarse dentro de un único marco matemático para resolver problemas de razonamiento abstracto.

Apéndice C. Multitensores

En este apéndice describimos con detalle la estructura de datos denominada **multitensor**, utilizada internamente por CompressARC.

Los multitensores constituyen el núcleo de la arquitectura propuesta y permiten representar relaciones entre diferentes subconjuntos de las dimensiones presentes en un rompecabezas de ARC-AGI.

C.1 Motivación

Una representación convencional almacena toda la información en un único tensor de alta dimensionalidad.

Por ejemplo, podríamos utilizar un tensor con dimensiones:

```
[ejemplo,
 color,
 anchura,
 altura,
 canal]
```

Sin embargo, esta representación presenta varios inconvenientes.

Muchas relaciones importantes únicamente dependen de un pequeño subconjunto de dichas dimensiones.

Por ejemplo:

- la relación entre un color y una columna;
- la posición de una línea horizontal;
- la correspondencia entre distintos ejemplos;
- la distribución de colores.

Obligar a que toda esta información se almacene dentro de un único tensor dificulta el aprendizaje y obliga a la red neuronal a redescubrir continuamente dichas relaciones.

Para evitar este problema, CompressARC utiliza una colección de tensores independientes.

Cada tensor representa únicamente un subconjunto concreto de dimensiones.

A esta colección de tensores la denominamos **multitensor**.

C.2 Definición formal

Sea

```
D = {
 ejemplo,
 color,
 anchura,
 altura
}
```

el conjunto de dimensiones disponibles.

Para cada subconjunto

$S \subseteq D$

se define un tensor independiente

T_S

cuyas dimensiones corresponden exactamente a los elementos de dicho subconjunto, junto con una dimensión adicional correspondiente a los canales.

Por ejemplo:

$T_{\emptyset}$

representa un único vector global.

Mientras que

$T_{\{\text{color}\}}$

contiene un vector para cada color.

Del mismo modo,

$T_{\{\text{anchura}, \text{altura}\}}$

almacena un vector independiente para cada posición espacial de la cuadrícula.

En conjunto, todos estos tensores forman el multitensor.

C.3 Número de tensores

Dado que existen cuatro dimensiones principales,

$2^4 = 16$

subconjuntos diferentes.

Por tanto, el multitensor contiene dieciséis tensores distintos.

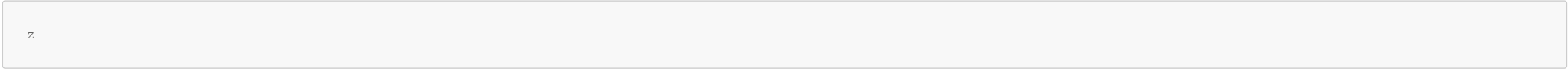
Estos tensores representan todos los niveles posibles de abstracción.

Por ejemplo:

- información completamente global;
- información asociada únicamente a un ejemplo;
- únicamente a un color;
- únicamente a una posición;
- combinaciones entre varias dimensiones.

C.4 Inicialización

El tensor latente



se genera directamente en formato multitensor.

Cada uno de los dieciséis tensores se inicializa mediante una distribución normal independiente.

Durante el entrenamiento, cada tensor aprende únicamente la información necesaria para describir el rompecabezas.

Esto produce una representación extremadamente eficiente.

Muchos tensores terminan convergiendo hacia distribuciones prácticamente idénticas a la normal estándar y, por tanto, apenas contribuyen a la divergencia KL.

En consecuencia, únicamente unos pocos tensores almacenan realmente información relevante.

C.5 Compartición de información

Las operaciones utilizadas por CompressARC permiten transferir información entre distintos tensores del multitensor.

Por ejemplo, una operación puede:

- leer información desde un tensor definido sobre colores;
- combinarla con otro tensor definido sobre posiciones;
- escribir posteriormente el resultado sobre el estado residual.

Esta forma de comunicación permite construir representaciones muy ricas sin necesidad de aumentar significativamente el número de parámetros.

C.6 Ventajas

Los autores destacan varias ventajas importantes del formato multitensor.

Representaciones más compactas

Cada concepto puede almacenarse únicamente sobre las dimensiones donde realmente resulta necesario.

Esto reduce considerablemente la redundancia.

Mejor sesgo inductivo

El diseño obliga implícitamente a la red neuronal a separar conceptos diferentes.

Por ejemplo:

- información espacial;
- información cromática;
- información correspondiente a distintos ejemplos.

Esta separación facilita el aprendizaje.

Mayor interpretabilidad

Resulta relativamente sencillo inspeccionar cada tensor por separado y determinar qué tipo de información contiene.

Precisamente esta propiedad permitió realizar el análisis del espacio latente presentado en la Sección 5.

Escalabilidad

Aunque el número de tensores aumenta exponencialmente con el número de dimensiones disponibles,

cada tensor individual permanece relativamente pequeño.

En la práctica, el coste computacional continúa siendo manejable.

C.7 Relación con otras arquitecturas

Los multitensores presentan ciertas similitudes con otras representaciones utilizadas anteriormente.

Por ejemplo:

- tensores factorizados;
- representaciones jerárquicas;
- grafos heterogéneos;
- arquitecturas con múltiples escalas.

No obstante, CompressARC emplea una construcción mucho más sistemática.

Cada subconjunto posible de dimensiones posee su propia representación explícita.

Las operaciones de la red permiten mover información libremente entre dichos subconjuntos.

C.8 Papel dentro de CompressARC

El objetivo principal del multitensor consiste en facilitar la compresión.

Cuando una determinada regularidad únicamente depende de unas pocas dimensiones, puede almacenarse directamente en el tensor correspondiente.

Por ejemplo:

- una correspondencia entre colores y direcciones no necesita información espacial completa;
- la posición de una línea horizontal no depende del color;

- una propiedad global del rompecabezas puede almacenarse en el tensor vacío.

Gracias a esta organización, la información necesaria para describir el rompecabezas queda distribuida entre distintos niveles de abstracción.

Ello reduce la cantidad de información residual que debe codificarse mediante las semillas y, por tanto, disminuye la longitud total del programa.

C.9 Resumen

El multitensores constituye uno de los elementos fundamentales de CompressARC.

En lugar de utilizar una única representación monolítica, la arquitectura mantiene simultáneamente una familia completa de representaciones especializadas.

Cada una captura regularidades asociadas únicamente a un subconjunto concreto de dimensiones.

Esta organización proporciona:

- mejores sesgos inductivos;
- mayor interpretabilidad;
- una representación mucho más eficiente desde el punto de vista de la compresión de información.

Los siguientes apéndices describen las operaciones concretas que actúan sobre estos multitensores y la implementación detallada de la arquitectura neuronal.

Apéndice D. Detalles de la arquitectura

En este apéndice describimos con precisión todas las operaciones que forman la arquitectura neuronal de CompressARC.

Aunque la estructura general de la red es relativamente sencilla, gran parte de su capacidad proviene del diseño específico de estas operaciones y de la manera en que intercambian información entre los distintos tensores del multitensores.

D.1 Flujo residual

Al igual que en los Transformers modernos, CompressARC mantiene un **estado residual** que atraviesa toda la red.

Cada bloque realiza siempre el mismo patrón de operaciones:

1. Leer información del estado residual.
2. Transformarla mediante una operación especializada.
3. Escribir el resultado de nuevo sobre el estado residual.
4. Sumar la actualización mediante una conexión residual.

Gracias a esta estructura, diferentes operaciones pueden cooperar sin destruir la información previamente aprendida.

D.2 Proyecciones lineales

Cada operación especializada está rodeada por dos transformaciones lineales:

- una proyección de entrada;
- una proyección de salida.

La primera adapta la representación residual al formato requerido por la operación.

La segunda vuelve a proyectar el resultado hacia el espacio residual común.

Es importante destacar que las propias operaciones especializadas prácticamente no contienen parámetros entrenables.

La mayor parte de los parámetros del modelo se concentra precisamente en estas proyecciones lineales.

D.3 Operación Broadcast (Difusión)

La operación **Broadcast** permite compartir información entre tensores definidos sobre diferentes subconjuntos de dimensiones.

Su funcionamiento puede resumirse en dos pasos.

Paso 1. Reducción

Primero se agregan valores eliminando una o varias dimensiones mediante operaciones de suma.

Por ejemplo, un tensor definido sobre:

```
[color, anchura]
```

puede reducirse a uno definido únicamente sobre:

```
[color]
```

sumando todos los valores correspondientes a las distintas columnas.

Paso 2. Difusión

Posteriormente el resultado se replica sobre nuevas dimensiones hasta alcanzar el formato requerido.

Por ejemplo:

```
[color]
```

puede expandirse nuevamente hasta

```
[color, anchura]
```

copiando el mismo vector en todas las columnas.

Esta operación permite construir fácilmente información global o compartir características entre distintas partes del rompecabezas.

D.4 Softmax

CompressARC utiliza funciones Softmax sobre distintas dimensiones del multitensores.

Dependiendo del contexto, Softmax puede aplicarse:

- sobre colores;
- sobre posiciones espaciales;
- sobre ejemplos;
- sobre cualquier otra dimensión disponible.

Esto permite generar distribuciones de probabilidad adaptadas a diferentes tipos de razonamiento.

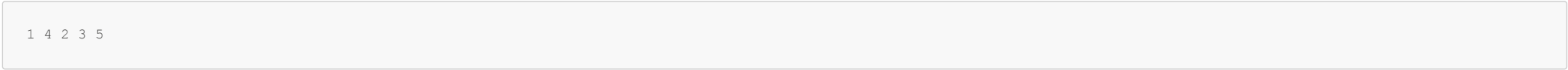
Por ejemplo, una Softmax aplicada sobre los colores puede representar directamente la probabilidad de cada color para un píxel determinado.

D.5 Máximo acumulado (Cumulative Max)

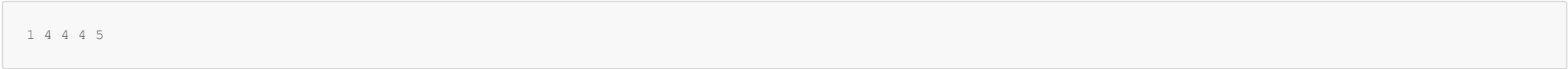
La operación **Cumulative Max** calcula máximos acumulados a lo largo de una dimensión.

Formalmente, cada posición contiene el máximo observado hasta ese punto.

Por ejemplo, dada la secuencia:



el máximo acumulado produce:



Este operador introduce una direccionalidad que resulta muy útil para numerosos rompecabezas donde intervienen:

- crecimiento de objetos;
- propagación;
- expansión;
- detección de bordes.

Además, rompe deliberadamente algunas simetrías espaciales que de otro modo resultarían difíciles de aprender.

D.6 Desplazamiento (Shift)

La operación **Shift** desplaza un tensor exactamente un píxel en una dirección determinada.

Puede aplicarse sobre cualquiera de las dimensiones espaciales.

Los desplazamientos posibles incluyen:

- arriba;
- abajo;
- izquierda;
- derecha.

Los valores que quedan fuera de la cuadrícula se rellenan con ceros.

Esta operación proporciona una forma muy eficiente de modelar transformaciones espaciales frecuentes en ARC-AGI.

Muchos rompecabezas consisten precisamente en trasladar objetos manteniendo inalterada su forma.

D.7 Funciones no lineales

Después de determinadas operaciones se aplica una función de activación elemento a elemento.

Estas funciones permiten introducir relaciones no lineales entre las distintas características aprendidas.

Su comportamiento es completamente independiente para cada componente del tensor.

D.8 Normalización

CompressARC utiliza capas de normalización aplicadas sobre la dimensión correspondiente a los canales.

El objetivo consiste en mantener una escala numérica estable durante el entrenamiento.

Ello facilita la optimización mediante descenso por gradiente y evita que determinadas operaciones dominen al resto.

D.9 Bloque residual completo

Cada bloque residual sigue la siguiente secuencia general:

1. Normalización.
2. Proyección lineal.
3. Operación especializada.
4. Proyección lineal de salida.
5. Suma residual.

Este patrón se repite para todas las operaciones presentes dentro del núcleo de la arquitectura.

Como resultado, el estado residual va acumulando progresivamente representaciones cada vez más útiles para reconstruir el rompecabezas.

D.10 Repetición de bloques

El núcleo completo se repite cuatro veces.

Los autores observaron experimentalmente que un número reducido de bloques resulta suficiente para capturar las regularidades presentes en ARC-AGI.

Incrementar significativamente la profundidad apenas mejoraba la precisión, mientras que aumentaba considerablemente el tiempo de entrenamiento durante la inferencia.

Por este motivo se optó por una arquitectura relativamente poco profunda pero cuidadosamente diseñada.

Apéndice D (continuación). Detalles de la arquitectura

D.11 Composición de operaciones

Una característica importante de la arquitectura es que ninguna operación especializada pretende resolver por sí sola un rompecabezas completo.

Cada operación únicamente introduce una capacidad elemental.

Por ejemplo:

- desplazar información;
- propagar información;
- seleccionar máximos;
- combinar dimensiones;
- normalizar representaciones.

La inteligencia emerge de la composición repetida de estas operaciones dentro del flujo residual.

Esta filosofía recuerda al diseño de los Transformers, donde la atención y las redes feed-forward son relativamente simples de manera individual, pero generan un comportamiento complejo al combinarse.

D.12 Escritura sobre el estado residual

Después de ejecutar una operación especializada, el resultado nunca sustituye completamente al estado residual.

En su lugar, se proyecta nuevamente sobre el espacio residual y se suma a la representación existente.

Formalmente:

```
Residual ← Residual + Projection( Operation( Projection( Residual ) ) )
```

Este mecanismo permite que distintas operaciones colaboren sin destruir la información producida por las anteriores.

Como consecuencia, el entrenamiento resulta considerablemente más estable.

D.13 Dimensión de canales

La dimensión correspondiente a los canales constituye el único espacio donde realmente se realizan transformaciones aprendidas.

Todas las proyecciones lineales actúan exclusivamente sobre esta dimensión.

Las dimensiones estructurales del multitenzor (ejemplo, color, anchura y altura) permanecen intactas durante dichas transformaciones.

Esta separación entre:

- estructura;
- representación;

simplifica notablemente la arquitectura.

D.14 Distribución de parámetros

La mayor parte de los aproximadamente **76 000 parámetros** del modelo se concentra en:

- proyecciones lineales de entrada;
- proyecciones lineales de salida.

En cambio:

- Broadcast;
- Shift;
- Cumulative Max;
- Softmax;
- Normalización;

aportan muy pocos o ningún parámetro entrenable.

Esto implica que la capacidad del modelo depende mucho más del diseño de sus operaciones que del número de parámetros.

D.15 Eficiencia computacional

Muchas de las operaciones utilizadas recorren únicamente una dimensión del multitenzor.

En consecuencia, su coste computacional crece aproximadamente de forma lineal con el tamaño de dicha dimensión.

Esto permite entrenar la red durante la inferencia incluso para rompecabezas relativamente grandes.

No obstante, el tiempo total continúa aumentando con el número de píxeles presentes en la cuadrícula.

Por ello los autores consideran que futuras implementaciones optimizadas podrían ofrecer mejoras significativas de velocidad.

D.16 Implementación vectorizada

Todas las operaciones fueron implementadas utilizando tensores completamente vectorizados.

No se realizan bucles explícitos sobre los píxeles del rompecabezas.

Este enfoque permite aprovechar eficientemente las operaciones masivas disponibles en aceleradores como GPU.

Además, simplifica la diferenciación automática utilizada por el descenso por gradiente.

D.17 Inicialización

Todos los pesos de la red neuronal se inician aleatoriamente al comienzo de cada rompecabezas.

No existe ningún conocimiento compartido entre rompecabezas diferentes.

Esto significa que, al comenzar la optimización de un nuevo rompecabezas, la red se encuentra exactamente en el mismo estado que tendría un modelo completamente nuevo.

Toda la información adquirida durante el entrenamiento anterior se descarta.

D.18 Optimización

Los parámetros se optimizan utilizando el algoritmo **Adam**.

Durante cada iteración se actualizan simultáneamente:

- los parámetros de la distribución latente;
- los pesos de todas las proyecciones lineales.

La arquitectura permanece fija durante todo el proceso.

D.19 Decisiones de diseño

Los autores destacan varias decisiones adoptadas tras numerosos experimentos preliminares.

Entre ellas:

- mantener una arquitectura relativamente pequeña;
- favorecer operaciones con significado geométrico;
- evitar introducir demasiados parámetros;
- preservar las simetrías presentes en ARC-AGI siempre que fuera posible.

Según los autores, estas decisiones permiten obtener una excelente relación entre capacidad de generalización y longitud del programa.

D.20 Interpretación

Desde el punto de vista del principio de Longitud Mínima de Descripción, la arquitectura constituye una parte fija del programa.

Cuanto mejor sea dicha arquitectura describiendo regularidades habituales de ARC-AGI, menos información adicional deberán almacenar:

- los pesos entrenados;
- la distribución latente;
- las semillas pseudoaleatorias.

Por ello, el diseño manual de buenos sesgos inductivos resulta equivalente a reducir el tamaño fijo del programa.

Resumen del Apéndice D

El diseño de CompressARC sigue una filosofía claramente distinta a la de los grandes modelos de lenguaje.

En lugar de incrementar continuamente el número de parámetros, los autores dedican el esfuerzo al diseño de operaciones cuidadosamente elegidas para manipular directamente las estructuras geométricas presentes en ARC-AGI.

El resultado es una arquitectura muy pequeña, altamente interpretable y especialmente adecuada para ser optimizada mediante el principio de Longitud Mínima de Descripción.

Las operaciones descritas en este apéndice constituyen la base sobre la que posteriormente se apoyan todos los experimentos presentados en el artículo.

Apéndice E. Detalles experimentales

En este apéndice se describen los detalles de implementación utilizados para todos los experimentos presentados en el artículo.

El objetivo es facilitar la reproducibilidad de los resultados obtenidos por CompressARC.

E.1 Inicialización

Cada rompecabezas comienza exactamente desde el mismo estado inicial.

Concretamente:

- todos los pesos de la red neuronal se inician aleatoriamente;
- la distribución latente comienza próxima a una distribución normal estándar;
- no se reutiliza ningún parámetro aprendido anteriormente.

Esto garantiza que cada rompecabezas constituye un proceso de aprendizaje completamente independiente.

E.2 Optimizador

Todos los experimentos utilizan el optimizador **Adam**.

Durante cada iteración se optimizan simultáneamente:

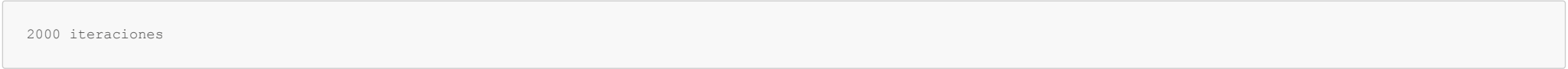
- los pesos de la red;
- los parámetros de la distribución latente.

La arquitectura permanece completamente fija.

No se modifica ninguna operación del modelo durante el entrenamiento.

E.3 Número de iteraciones

Cada rompecabezas se optimiza durante aproximadamente:



Este número representa un equilibrio entre:

- precisión;
- coste computacional.

Los autores observaron que aumentar considerablemente el número de iteraciones produce mejoras adicionales, aunque cada vez menores.

E.4 Tiempo de ejecución

El tiempo empleado depende principalmente de:

- el tamaño de la cuadrícula;
- el número de ejemplos disponibles;
- la complejidad del rompecabezas.

En promedio, cada rompecabezas requiere alrededor de:

20 minutos

en el hardware utilizado por los autores.

E.5 Criterio de parada

No se emplea un criterio dinámico de parada temprana (*early stopping*).

Todos los rompecabezas reciben exactamente el mismo presupuesto de optimización.

Esta decisión simplifica la comparación entre experimentos.

E.6 Muestreo de soluciones

Una vez finalizado el entrenamiento durante la inferencia, CompressARC genera las soluciones directamente a partir de la distribución aprendida.

Cada intento corresponde a una muestra obtenida del modelo.

Como ocurre en la evaluación oficial de ARC-AGI, se permiten dos intentos por rompecabezas.

E.7 Reproducibilidad

Todos los experimentos utilizan:

- la misma arquitectura;
- el mismo procedimiento de entrenamiento;
- los mismos hiperparámetros.

No se ajustan parámetros específicos para rompecabezas individuales.

Apéndice F. Estudios de ablación

Con el fin de comprender qué componentes resultan realmente importantes, los autores realizan diversos estudios de ablación.

En cada experimento eliminan o modifican uno de los componentes principales de CompressARC y vuelven a medir la precisión obtenida.

F.1 Eliminación de la equivarianza

La primera ablación consiste en eliminar las propiedades de equivarianza de la arquitectura.

En este caso la red deja de tratar de manera equivalente:

- diferentes permutaciones de colores;
- rotaciones;
- reflexiones;
- cambios en el orden de los ejemplos.

Los resultados muestran una disminución apreciable del rendimiento.

Esto confirma que incorporar explícitamente las simetrías presentes en ARC-AGI constituye uno de los principales sesgos inductivos del modelo.

F.2 Eliminación del multitenzor

En esta variante el multitenzor se sustituye por una representación convencional basada en un único tensor.

La precisión disminuye de forma significativa.

Los autores concluyen que distribuir la información entre distintos subconjuntos de dimensiones facilita considerablemente la compresión del rompecabezas.

F.3 Eliminación del término KL

Otra ablación consiste en eliminar completamente el término correspondiente a la divergencia KL.

En este caso la red únicamente intenta reconstruir perfectamente los ejemplos conocidos.

El resultado es un fuerte sobreajuste (*overfitting*).

Aunque la reconstrucción de los ejemplos de entrenamiento resulta excelente, la calidad de las soluciones generadas para los ejemplos de prueba disminuye notablemente.

Este experimento pone de manifiesto que el término KL actúa realmente como una presión para encontrar descripciones más simples.

F.4 Sustitución de las operaciones especializadas

Los autores sustituyen varias de las operaciones diseñadas específicamente para ARC-AGI por transformaciones lineales convencionales.

La precisión vuelve a reducirse.

Ello indica que operaciones como:

- Shift;
- Broadcast;
- Cumulative Max;

aportan capacidades difíciles de reproducir mediante capas densas estándar.

F.5 Incremento del número de parámetros

También se evalúan modelos considerablemente más grandes.

Sorprendentemente, aumentar el número de parámetros no produce mejoras proporcionales.

En algunos casos incluso empeora la capacidad de generalización.

Según los autores, esto ocurre porque modelos excesivamente grandes pueden memorizar con demasiada facilidad los ejemplos conocidos sin descubrir representaciones realmente compactas.

F.6 Conclusión de las ablaciones

Los estudios realizados muestran que prácticamente todos los componentes principales de CompressARC contribuyen al rendimiento final.

En particular destacan tres elementos fundamentales:

1. la formulación basada en Longitud Mínima de Descripción;
2. la arquitectura equivariante;
3. la representación mediante multitensores.

La combinación de estos tres componentes resulta considerablemente más eficaz que cualquiera de ellos por separado.

Apéndice G. Análisis cualitativo de los resultados

Además de las métricas cuantitativas presentadas en el artículo principal, los autores realizaron un análisis detallado de los rompecabezas resueltos y no resueltos con el fin de identificar patrones comunes.

En lugar de limitarse a medir la precisión global, estudiaron qué tipos de transformaciones aprende realmente CompressARC.

Este análisis permite comprender mejor tanto las fortalezas como las limitaciones actuales del método.

G.1 Rompecabezas resueltos correctamente

CompressARC obtiene buenos resultados en problemas donde la solución puede describirse mediante regularidades relativamente compactas.

Entre los ejemplos más habituales se encuentran:

- propagación de colores;
- detección de regiones;
- prolongación de patrones cortos;
- copias y desplazamientos de objetos;
- simetrías simples;
- identificación de componentes conectados;
- transformaciones geométricas locales.

En muchos de estos casos la representación aprendida resulta fácilmente interpretable mediante el análisis del espacio latente.

G.2 Errores frecuentes

Los fallos observados presentan varios patrones comunes.

En algunos rompecabezas la red identifica correctamente la regla general, pero comete pequeños errores en posiciones concretas.

En otros casos produce una solución casi correcta, aunque uno o varios objetos aparecen desplazados algunos píxeles.

También existen ejemplos donde la estructura global resulta correcta, pero determinados colores son intercambiados.

Estos errores sugieren que la arquitectura ha aprendido una representación próxima a la solución correcta, aunque todavía insuficientemente precisa.

G.3 Evolución durante el entrenamiento

Uno de los aspectos más interesantes observados por los autores es que la calidad de las soluciones no mejora siempre de forma monotónica.

Durante el entrenamiento aparecen fases donde:

- el error disminuye rápidamente;
- posteriormente aumenta;
- vuelve a reducirse más adelante.

Este comportamiento coincide con las variaciones observadas en la divergencia KL.

Los autores interpretan estos cambios como reorganizaciones internas de la representación latente.

Apéndice H. Capacidades y limitaciones

Este apéndice resume qué habilidades parecen emerger de la arquitectura propuesta y cuáles continúan representando un desafío.

Los autores insisten en que esta clasificación procede de la observación empírica de numerosos rompecabezas y no constituye una lista exhaustiva.

H.1 Capacidades observadas

CompressARC demuestra ser capaz de aprender tareas como:

- detectar objetos;
- identificar componentes conectados;
- propagar patrones;
- copiar regiones;
- trasladar objetos;
- completar figuras parcialmente visibles;
- aplicar simetrías sencillas;
- asociar colores con posiciones;
- utilizar información procedente de varios ejemplos simultáneamente;
- aprender representaciones geométricas compactas.

En muchos casos estas habilidades emergen únicamente durante la optimización realizada en tiempo de inferencia.

H.2 Limitaciones observadas

Por el contrario, los autores identifican varias limitaciones importantes.

Entre ellas destacan:

- dificultades para realizar razonamientos que requieren muchas etapas consecutivas;
- propagaciones a grandes distancias;
- dependencias globales muy complejas;
- conteos exactos de numerosos objetos;
- algoritmos iterativos largos;
- reglas que requieren memoria de largo alcance.

Estos problemas aparecen principalmente cuando la solución exige mantener información durante muchas transformaciones sucesivas.

H.3 Influencia de la arquitectura

Muchas de las limitaciones anteriores no parecen deberse al principio de Longitud Mínima de Descripción.

Según los autores, la causa principal reside en la capacidad limitada de la arquitectura neuronal utilizada.

Por ejemplo, una operación Shift únicamente desplaza la información un píxel por capa.

Con cuatro bloques residuales, la distancia máxima sobre la que puede propagarse información resulta relativamente pequeña.

Este hecho explica por qué algunos patrones dejan de extenderse correctamente cuando deben recorrer grandes regiones de la cuadrícula.

H.4 Posibles mejoras

Los autores proponen varias líneas para superar estas limitaciones.

Entre ellas:

- arquitecturas más profundas;
- nuevas operaciones especializadas;
- implementaciones más eficientes que permitan ejecutar muchas más iteraciones;
- mecanismos de propagación de largo alcance;
- mejores sesgos inductivos.

Esperan que estas mejoras aumenten considerablemente la capacidad del enfoque sin abandonar el marco basado en MDL.

Apéndice I. Análisis de las representaciones aprendidas

Una de las ventajas más importantes de CompressARC consiste en que sus representaciones internas resultan relativamente fáciles de interpretar.

A diferencia de muchos modelos de gran tamaño, los distintos componentes del multitensor pueden inspeccionarse individualmente.

I.1 Distribución de la información

Los autores observan que la mayor parte de los tensores terminan almacenando muy poca información.

Únicamente unos pocos contribuyen significativamente a la divergencia KL.

Esto indica que la representación aprendida es altamente dispersa (*sparse*).

La mayor parte de la estructura del rompecabezas queda concentrada en unos pocos tensores especializados.

I.2 Especialización espontánea

Durante el entrenamiento distintos tensores adquieren funciones claramente diferenciadas.

Por ejemplo:

- algunos representan relaciones espaciales;
- otros almacenan asociaciones entre colores;
- otros describen posiciones características;
- otros contienen información prácticamente global.

Esta especialización aparece espontáneamente como consecuencia de la optimización de la longitud del programa.

No se introduce explícitamente mediante ninguna restricción adicional.

I.3 Interpretabilidad

La posibilidad de inspeccionar cada tensor por separado facilita enormemente comprender el comportamiento del modelo.

Los autores consideran que esta interpretabilidad constituye una ventaja importante frente a arquitecturas mucho mayores donde la información aparece distribuida de forma mucho más difusa.

Además, permite diagnosticar con relativa facilidad por qué determinados rompecabezas no llegan a resolverse correctamente.

Apéndice J. Ejemplos adicionales

En este apéndice se presentan numerosos ejemplos adicionales de rompecabezas pertenecientes a ARC-AGI, junto con las soluciones producidas por CompressARC.

El objetivo es ilustrar visualmente el comportamiento del sistema en una amplia variedad de situaciones.

Cada ejemplo muestra:

- los pares de entrenamiento;
- la entrada de prueba;
- la solución generada por CompressARC;
- la solución correcta.

Los ejemplos incluyen tanto casos resueltos correctamente como ejemplos en los que el modelo comete errores.

Esta comparación permite apreciar con mayor claridad qué tipos de razonamiento consigue realizar actualmente la arquitectura.

J.1 Soluciones correctas

En los ejemplos correctamente resueltos se observa que CompressARC suele descubrir representaciones compactas capaces de explicar simultáneamente todos los ejemplos disponibles.

En muchos casos la solución aparece de forma relativamente temprana durante la optimización y posteriormente se mantiene estable durante el resto del entrenamiento.

J.2 Soluciones parcialmente correctas

También aparecen numerosos casos donde la solución es casi correcta.

Entre los errores más frecuentes destacan:

- un único píxel incorrecto;
- colores intercambiados;

- pequeños desplazamientos;
- errores localizados en regiones alejadas del resto de la estructura.

Estos ejemplos muestran que la representación aprendida suele capturar correctamente la regla principal, aunque determinados detalles permanecen sin resolver.

J.3 Casos no resueltos

Los rompecabezas completamente no resueltos suelen compartir alguna de las siguientes características:

- reglas muy largas;
- transformaciones altamente iterativas;
- dependencias globales complejas;
- necesidad de contar con precisión numerosos objetos;
- relaciones espaciales de largo alcance.

Estos casos coinciden con las limitaciones descritas previamente en el Apéndice H.

Apéndice K. Tablas completas de resultados

Este apéndice recopila los resultados numéricos completos obtenidos durante la evaluación.

Para cada rompecabezas se incluyen, entre otros datos:

- identificador del rompecabezas;
- número de iteraciones realizadas;
- tiempo de ejecución;
- longitud aproximada del programa;
- resultado obtenido;
- estado (correcto o incorrecto).

Las tablas completas permiten reproducir exactamente las estadísticas resumidas en el cuerpo principal del artículo.

K.1 Resultados sobre el conjunto de entrenamiento

La primera tabla contiene todos los rompecabezas pertenecientes al conjunto oficial de entrenamiento de ARC-AGI.

Puede observarse una variabilidad considerable entre rompecabezas.

Algunos convergen rápidamente hacia una solución correcta.

Otros requieren prácticamente todas las iteraciones disponibles.

También existen rompecabezas donde el entrenamiento no consigue encontrar una representación suficientemente compacta.

K.2 Resultados sobre el conjunto de evaluación

La segunda tabla presenta los resultados correspondientes al conjunto oficial de evaluación.

La precisión global obtenida coincide con los valores resumidos anteriormente.

Los autores incluyen igualmente los tiempos de ejecución individuales para facilitar futuras comparaciones con otros métodos.

K.3 Observaciones

Las tablas muestran que el coste computacional depende principalmente del tamaño del rompecabezas y no únicamente de su dificultad conceptual.

Algunos problemas conceptualmente sencillos requieren tiempos relativamente elevados simplemente porque contienen cuadrículas de mayor tamaño.

Apéndice L. Conclusiones finales

CompressARC demuestra que es posible abordar problemas de razonamiento abstracto utilizando un paradigma completamente diferente al dominante en los modelos actuales de gran escala.

En lugar de depender de enormes cantidades de datos de entrenamiento y de modelos con miles de millones de parámetros, el sistema:

- comienza desde parámetros aleatorios;
- aprende exclusivamente durante la inferencia;
- optimiza directamente una aproximación diferenciable de la longitud del programa que describe el rompecabezas.

Los experimentos muestran que este enfoque permite resolver una proporción significativa de los rompecabezas de ARC-AGI sin utilizar ningún conocimiento adquirido previamente.

Perspectiva

Los autores consideran que CompressARC representa únicamente un primer paso.

Existen numerosas posibilidades para mejorar el sistema mediante:

- arquitecturas más expresivas;
- operaciones especializadas adicionales;
- implementaciones más eficientes;
- mejores estrategias de optimización.

Todas estas mejoras podrían incorporarse manteniendo intacta la idea fundamental del trabajo:

Resolver un problema equivale a encontrar la descripción computacional más corta capaz de generarlo.

Mensaje principal del artículo

La principal aportación de CompressARC no consiste únicamente en el porcentaje de precisión alcanzado.

Su contribución más importante es demostrar experimentalmente que el principio de **Longitud Mínima de Descripción (Minimum Description Length, MDL)** puede utilizarse como objetivo de entrenamiento directo para redes neuronales entrenadas íntegramente durante la inferencia.

Este resultado abre una nueva línea de investigación situada entre:

- la teoría algorítmica de la información;
- la compresión de datos;
- el aprendizaje profundo;
- el razonamiento abstracto.

Los autores sugieren que esta combinación podría constituir una alternativa prometedora —o al menos complementaria— al paradigma basado exclusivamente en el escalado masivo de modelos y datos.